

함수와 데이터다루기

인공지능 개론

Jinu Gong

학습 목표

- 함수를 정의하고, 호출하여 사용하여 방법을 학습해보자
- 함수에 일을 시키기 위해서 넘겨주는 인자와 매개변수의 차이점을 알아보자
- 데이터 분석 라이브러리인 판다스에 대해서 살펴보자
- 데이터프레임을 합하여 새로운 테이블을 만들어보자
- 판다스 데이터를 matplotlib으로 시각화하는 연습해보자

함수

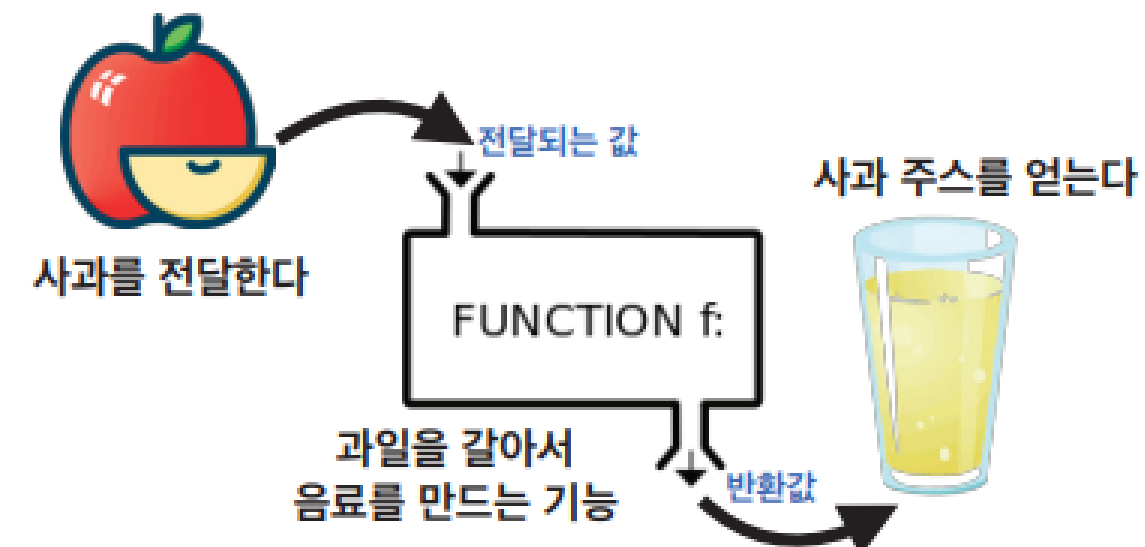
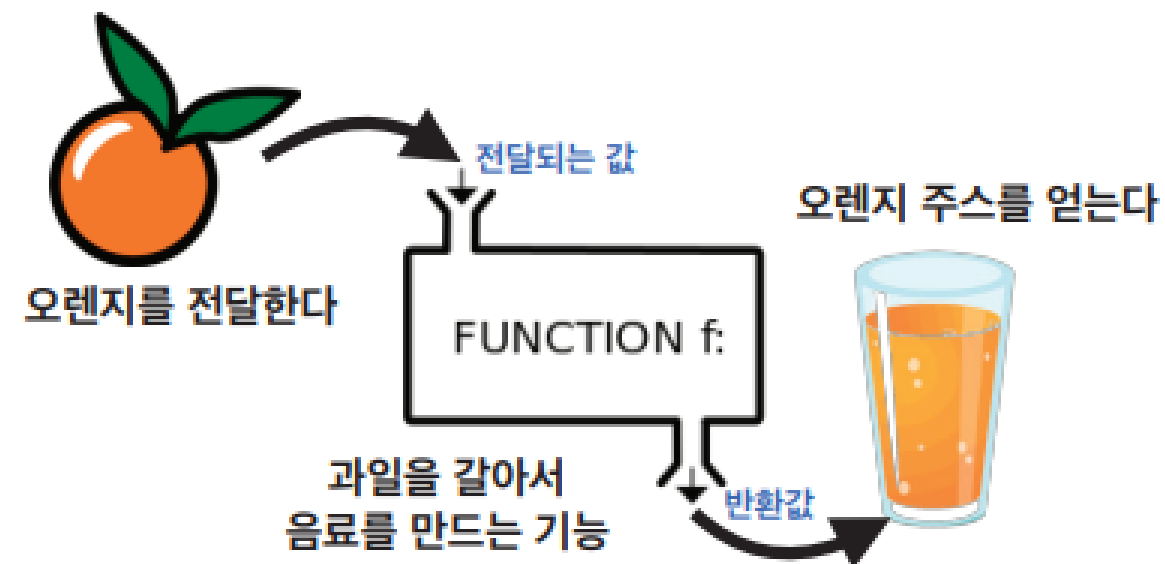
함수

- 파이썬에서는 프로그램을 쪼개는 3가지의 방법이 있다.
 - **함수**function 는 우리가 반복적으로 사용하는 코드를 묶은 것으로 코드의 덩어리와 같다.
 - **객체**object 는 코드 중에서 독립적인 단위로 분리할 수 있는 조각이다.
 - **모듈**module 은 프로그램의 일부를 가지고 있는 독립적인 파일이다.
- **함수**는 일을 수행하는 코드의 덩어리이며, 더 큰 프로그램을 구축하는 데 사용할 수 있는 작은 조각이다.



함수

- 함수는 그림과 같이 입력을 받아서 출력을 내보내는 블랙박스로 생각할 수 있다. 아래 그림은 과일을 갈아서 음료로 만드는 기능을 가진 블랙박스이다.
- 이때 오렌지 주스를 만들기 위해서는 오렌지를 입력으로 전달하면 되고, 사과 주스를 만들기 위해서는 사과를 입력으로 전달하면 된다. 즉 이 블랙박스의 이용자는 주스를 만들기 위하여 블랙박스를 교체하지 않고 전달하는 과일만 바꾸면 되는 것이다.



def 예약어를 이용하여 함수를 작성하고 호출하기

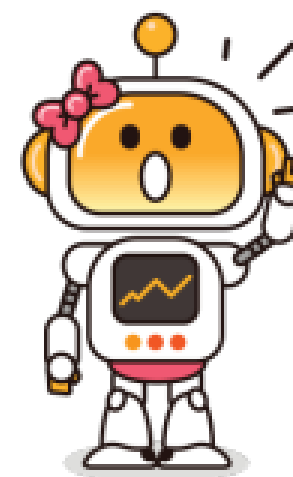
- 첫째 줄 `def print_address():`에서 `def` 키워드를 이용하여 함수를 정의하고 있다. 이어서 함수의 이름을 적은 후에 소괄호 `()`와 콜론 `:`을 붙인다. 여기서 콜론 `:`은 코드 블록이 이어서 등장한다는 것을 의미한다. 이것은 이전 장에서 배운 `if`, `for`, `while`의 경우와 마찬가지로이다.
- 두 번째 줄부터 네 번째 줄까지는 함수를 구성하는 문장들이 들어간다. 함수를 구성하는 문장들은 반드시 들여쓰기를 해야 한다. 이 덩어리를 블록이라고 하고, 함수는 이 블록을 실행한다.

함수 정의를 위한 예약어 함수의 이름 함수가 받아오는 데이터: 이 함수는 매개변수가 없음

```
def print_address():  
    print('경상북도 울릉군 울릉읍')  
    print('독도리 산 1-96번지')
```

하위 블록의 시작 함수 몸체 : 들여쓰기 블록

하나의 블록을 이루는 코드 문치는 같은 깊이로 들여쓰여 함



함수를 정의하기 위해서는 `def`라는 예약어를 사용합니다. 다음으로 소괄호와 콜론을 붙입니다. 함수의 몸체는 들여쓰기를 한 코드 블록으로 구성합니다.

def 예약어를 이용하여 함수를 작성하고 호출하기

- 함수 안에 있는 코드들은 자동으로 실행되지 않으며, 함수가 호출되어야 함수 안의 코드가 실행된다.
- **함수 호출** function call이란 다음과 같이 함수 이름을 적어주는 것이다. 위의 코드에 다음과 같은 1줄만 추가하면 print_address() 함수가 실행되고 아래와 같이 실행 결과가 표시된다.

```
def print_address() :  
    print('경상북도 울릉군 울릉읍'  
    print('독도이사부길 63')  
  
print_address() # 정의한 함수를 호출
```

경상북도 울릉군 울릉읍
독도이사부길 63

함수 호출로
인한 수행결과.

반드시 함수 호출문이
있어야만 함수가 수행된다.

함수의 활용

- 우리가 함수를 정의하더라도 함수를 호출하지 않으면 함수 안의 코드는 실행되지 않는다.
- 함수를 호출하려면 함수 이름과 소괄호를 이용한다. 아래는 수행 결과

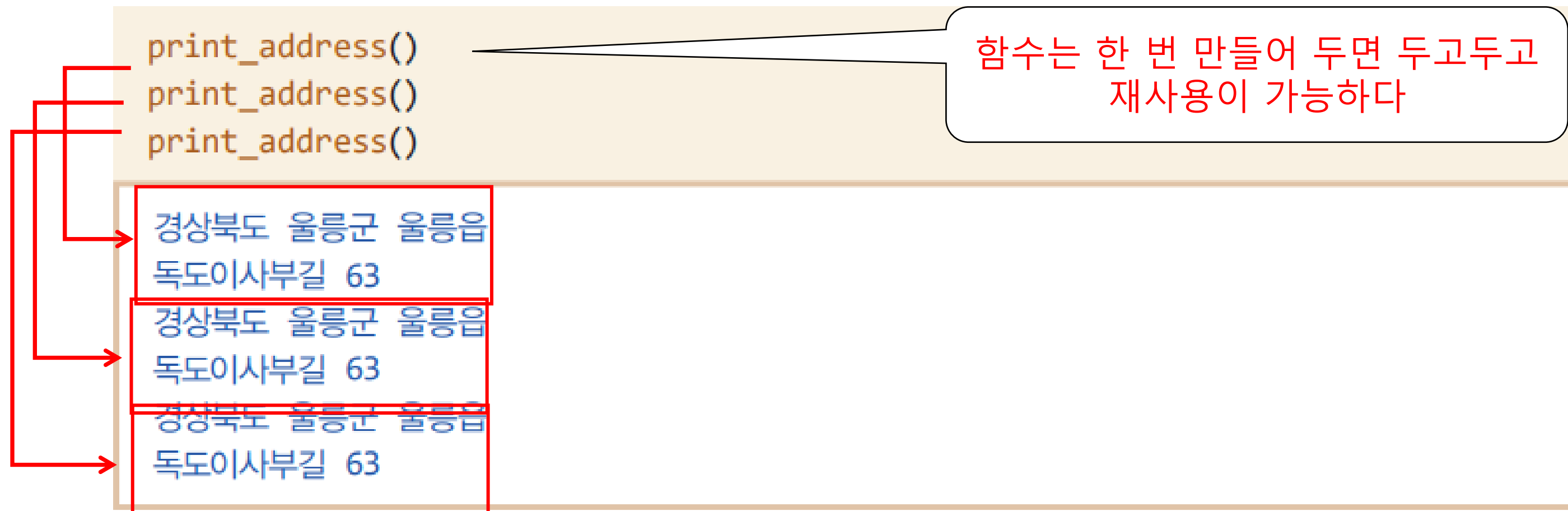
```
경상북도 울릉군 울릉읍  
독도이사부길 63
```

- 그런데 생각해 보면 위와 같은 출력은 아래와 같은 문장으로도 얼마든지 출력할 수 있다.

```
print('경상북도 울릉군 울릉읍')  
print('독도이사부길 63')
```

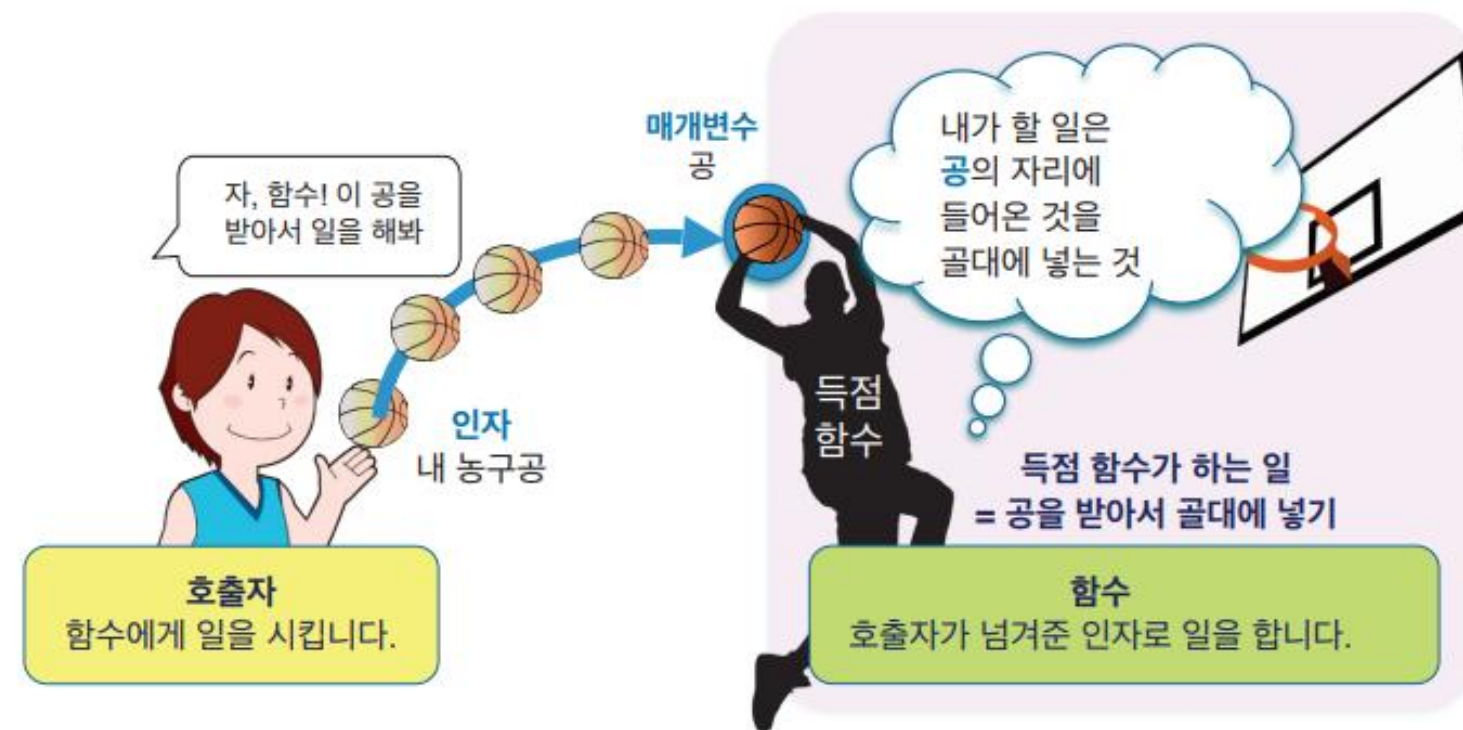

함수의 활용

- 함수를 사용하는 주된 이유는 우리가 한 번만 함수를 정의하면 언제든지 필요할 때면 함수를 불러서 일을 시킬 수 있기 때문이다.
- 이 주소를 3번 인쇄하여 출력하고자 하는 경우를 생각해보자. 함수가 정의된 상태라면 우리는 다음과 같이 이 작업을 수행할 수 있다.



함수에 하나의 값을 넘겨주고 일을 시키자

- 우리는 함수 외부에서 함수 안으로 값(정보)을 전달할 수 있다. 이 값을 **인자argument**라고 한다.



- 예를 들어서 앞의 주소를 인쇄하는 `print_address()` 함수에서 수신자의 이름은 외부에서 전달받는 것으로 해보자. 수신자의 이름을 변경할 수 있으면 같은 회사에서 근무하는 사람들도 사용할 수 있을 것이다.

함수에 하나의 값을 넘겨주고 일을 시키자

```
def print_address(name):  
    print("서울 특별시 종로구 1번지")  
    print("파이썬 빌딩 7층")  
    print("수신 :", name)
```

함수의 매개변수 name을
통해서 그 결과를 다르게
만들 수 있다

함수에 전달되는 실제값으로
인자라고 함 : "홍길동" 대신
여러분의 이름도 가능하다

```
print_address("홍길동") # "홍길동" 대신 다른 이름을 넣을 수 있다  
print_address("넌넌한 교수")
```

- 함수 정의부를 살펴보면 이전과 달리 함수 이름 뒤의 소괄호 안에 변수 name이 있다. 이 변수 name을 통하여 함수로 값이 전달된다. 메인 프로그램에서 print_address()를 호출할 때 "홍길동"이라는 문자열을 괄호 안에 넣어 주었는데. 이것이 함수 내부의 name 변수로 전달되는 것이다.
- 함수 호출시 전달되는 실제 값을 **인자**argument라고 하고, 함수 내부에서 전달 받는 변수를 **매개변수**parameter라고 한다.

함수에 하나의 값을 넘겨주고 일을 시키자

함수 내부에서 전달 받는 변수 : 매개변수

```
def print_address(name):  
    print("서울 특별시 종로구 1번지")  
    print("파이썬 빌딩 7층")  
    print("수신 :", name)
```

```
print_address("홍길동") # "홍길동" 대신 다른 이름  
print_address("넌넌한 교수")
```

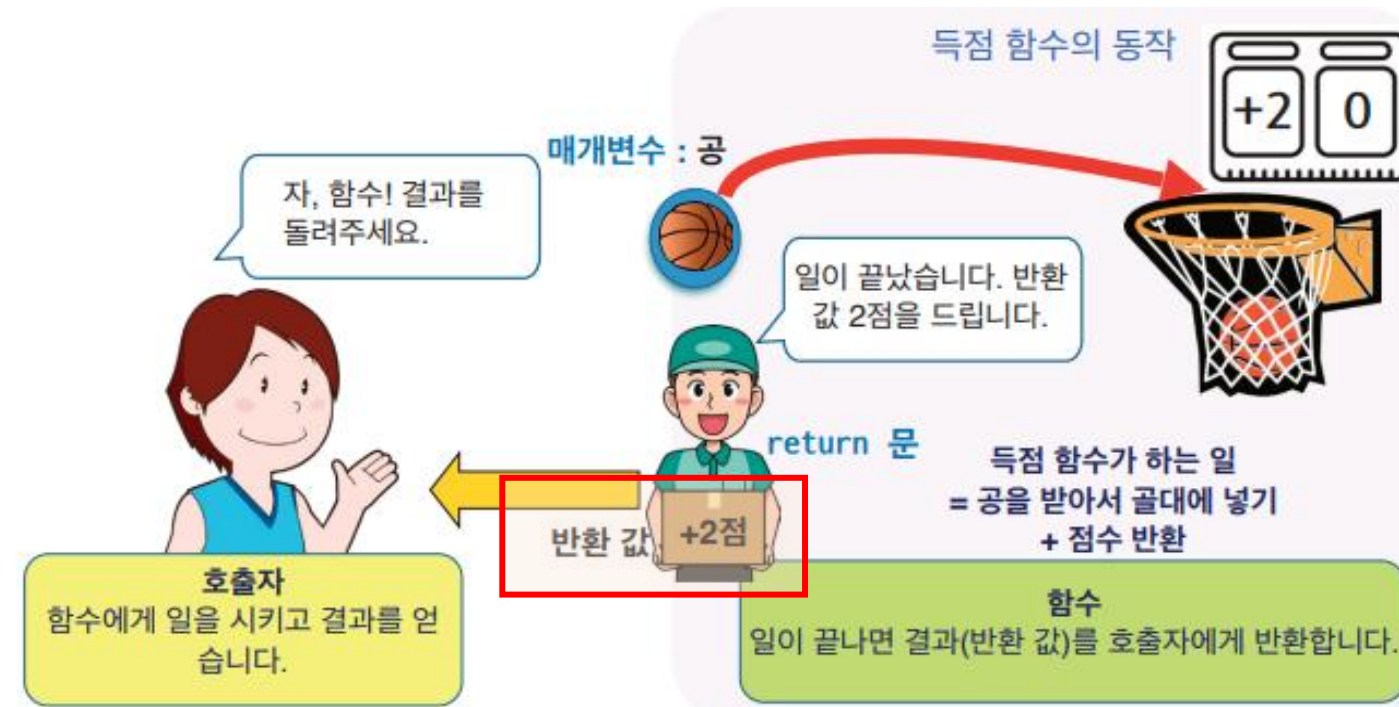
서울 특별시 종로구 1번지
파이썬 빌딩 7층
홍길동

서울 특별시 종로구 1번지
파이썬 빌딩 7층
넌넌한 교수

함수에 넘겨지는 값 : 인자

함수 값 받아오기

- 지금까지, 함수는 우리가 주는 값을 받아서 작업을 처리하였다. 그러나 함수가 매우 유용한 것은 함수가 주어진 일을 하고 우리에게 뭔가를 보낼 수 있다는 점이다. 이와 같이 함수로부터 되돌아오는 값을 **반환 값** return value이라고 한다.



- 함수가 함수 내부의 값을 외부에 보낼 때는 **return 키워드**를 사용하면 된다. 원의 반지름을 보내면 원의 면적을 계산해서 반환하는 함수를 작성해보자.

함수 값 받아오기

- 원의 반지름을 보내면 원의 면적을 계산해서 반환하는 함수를 작성해보자.

```
def calculate_area(radius):  
    area = 3.14 * radius**2    # 원의 면적을 구하는 근사식  
    return area                # 이전 줄에서 구한 area 값을 호출문에 돌려준다
```

- 이 함수의 호출문에 다음과 같은 할당 문을 넣어보자.

```
c_area = calculate_area(5.0)    # calculate_are() 함수가 계산한 값을 c_area에 저장
```

- c_area 에는 78.5 값이 반환되어 저장된다. 다음과 같이 출력해 보자.

```
>>> print(calculate_area(5.0))  
78.5  
>>> area_sum = calculate_area(5.0) + calculate_area(10.0)  
>>> print(area_sum)  
392.5
```

- 다음과 같이 하면 함수 호출만 하고 그 반환값을 사용하지 않게 된다.

```
>>> calculate_area(10.0)    # 함수를 호출만 하고 그 반환값을 사용안함  
314.0
```

함수 값 받아오기

- 2개의 정수 start와 end를 받아서 start에서 end까지의 합을 계산하는 함수 get_sum()을 작성하자.

```
def get_sum(start, end):  
    s = 0  
    for i in range(start, end+1):  
        s += i  
    return s  
  
x = get_sum(1, 10)  
print('1에서 10까지의 합 :', x)
```

start, end를 매개변수로하여 인자 값을 받는다
start부터 end까지 정수의 합을 구함
start부터 end까지 수의 합을 반환한다
인자 1, 10을 넘겨주며 함수 get_sum()을 호출함
1에서 10까지의 정수의 합 55를 출력한다

1에서 10까지의 합 : 55

- 인자와 매개변수는 함수 호출 시에 데이터를 주고받기 위하여 필요하다. 다시 정리하면, **인자**argument는 호출 프로그램에 의하여 함수에 실제로 전달되는 값이다. **매개변수**parameter는 이 값을 전달받는 변수이다.

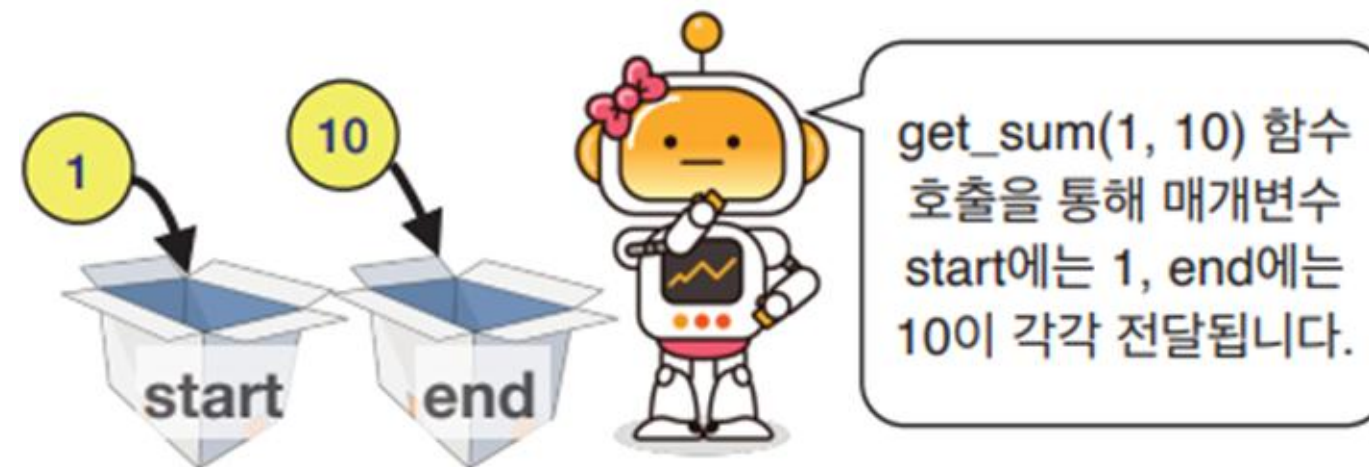
함수 값 받아오기

- 여기서 주의할 점은 매개변수의 개수와 넘어가는 인자의 개수가 정확히 일치하여야 한다는 점이다.
- 즉, 매개 변수가 두 개이면 인자도 두 개를 전달하여야 한다. 매개변수의 개수와 인자의 개수가 일치하지 않으면 오류가 발생하게 된다.

```
y = get_sum(1, 20)  
print('1에서 20까지의 합 :', y)
```

1과 20이 인자가 된다. y는 1에서 20까지의 합을 반환받는다.

1에서 20까지의 합 : 210



함수 값 받아오기

- 지금까지 함수는 호출될 때 맡은 일을 수행하고 수행을 마치거나, 수행을 마칠 때 **하나의 값을 반환**하는 일을 하는 것을 살펴보았다. 파이썬의 큰 특징 중 하나는 함수가 **여러 개의 값을 반환**할 수 있다는 것으로, 아래와 같은 형식을 사용한다. 가장 큰 이유는 생산성이 뛰어나기 때문이다.
- 이와 같은 특징 때문에 파이썬을 이용하면 간결하면서도 효율적인 프로그램을 빠르게 작성할 수 있다.

```
def sort_num(n1, n2):           # 2개의 값을 받아오는 함수
    if n1 < n2:                 # n1이 더 작으면 n1, n2 순서로 반환
        return n1, n2
    else:                       # n2가 더 작으면 n2, n1 순서로 반환
        return n2, n1

print(sort_num(110, 210))       # 110과 210을 함수의 인자로 전달하고 반환되는 값을 출력
print(sort_num(2100, 80))
```

(110, 210)
(80, 2100)

항상 작은 수, 큰 수 쌍이 반환된다

함수 값 받아오기

- 2개의 수를 인자로 받아서 사칙연산을 수행하고 그 값을 출력하는 프로그램

```
def calc(n1, n2):  
    return n1 + n2, n1 - n2, n1 * n2, n1 / n2  # 덧셈, 뺄셈, 곱셈, 나눗셈 결과를 반환  
  
n1, n2 = 200, 100  
t1, t2, t3, t4 = calc(n1, n2)  # 네 개의 값을 반환받기 위해 4개의 변수를 사용함  
print(n1, '+', n2, '=', t1)  
print(n1, '-', n2, '=', t2)  
print(n1, '*', n2, '=', t3)  
print(n1, '/', n2, '=', t4)
```

사칙연산의 결과를
반환받는다.

```
200 + 100 = 300  
200 - 100 = 100  
200 * 100 = 20000  
200 / 100 = 2.0
```

변수의 범위

- 이제 변수의 사용 범위를 함수와 관련해 깊이 있게 다루어 보려 한다. 다음 코드를 작성하고 수행해 보라.

```
def print_counter():  
    print('counter =', counter) # 함수 내부의 counter 값  
  
counter = 100  
print_counter()  
print('counter =', counter)    # 함수 외부의 counter 값
```

```
counter = 100  
counter = 100
```

- counter라는 변수를 함수 외부에서 생성한 후 100이라는 값을 할당하자. 그리고 함수 내부에서 이 변수를 출력하면 100이 출력된다. 당연히 함수 외부에서도 이 출력값은 100이 될 것이다.

변수의 범위

- 이제 다음과 같이 함수 내부에서 `counter = 200`과 같이 `counter`에 새로운 값을 할당해 보도록 하자. 함수 외부와 내부 모두 200이 출력될까?

```
def print_counter():  
    counter = 200  
    print('counter =', counter) # 함수 내부의 counter 값  
  
counter = 100  
print_counter()  
print('counter =', counter)    # 함수 외부의 counter 값  
  
counter = 200  
counter = 100
```

- `counter = 200`과 같은 할당이 이루어질 때 이 변수는 함수 안에서 새로 선언되고 생성된다.
- 이와 같이 함수 안에서 생성되는 변수를 **지역변수** `local variable`라고 한다. 이 지역변수 `counter`가 참조하는 값 200은 전역변수가 참조하는 값 100과 별개의 메모리에 있으며, 지역변수는 함수가 종료되면 사라진다. 따라서 `counter` 변수를 함수의 외부에서 출력할 경우 200이 아닌 100이 출력되게 되는 것이다.

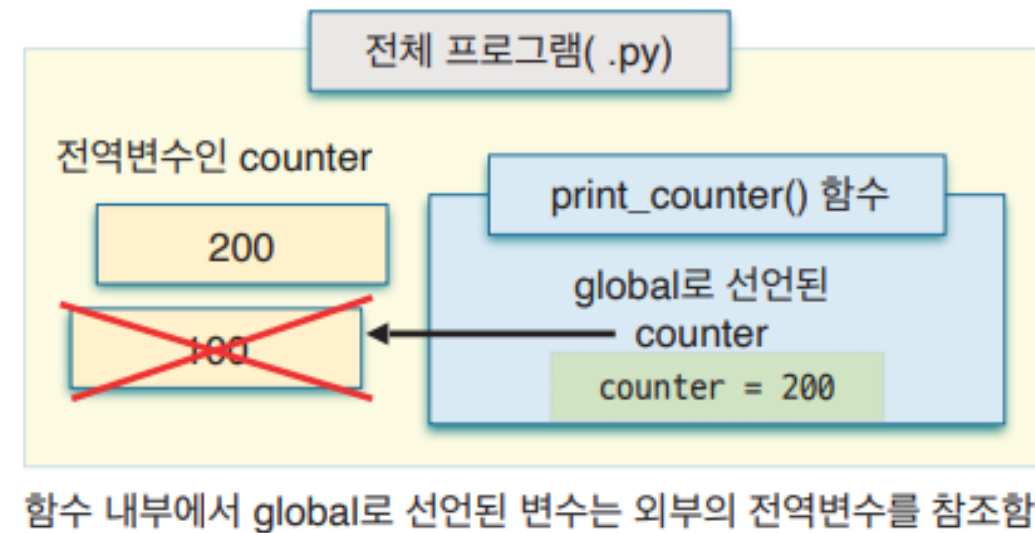
global 키워드

- 만일 함수 내부에서 새로 변수를 만들지 않고 전역변수인 counter를 불러서 사용하려면 어떻게 해야 할까? 아래와 같이 global이라는 키워드를 사용하는 것이 바람직하다. **global counter**는 함수 외부의 전역변수 counter를 사용하겠다는 선언이다.

```
def print_counter():  
    global counter      # 함수 외부의 전역변수 counter를 사용하겠다는 선언  
    counter = 200  
    print('counter =', counter) # 함수 내부의 counter 값  
  
counter = 100  
print_counter()  
print('counter =', counter)    # 함수 외부의 counter 값  
  
counter = 200  
counter = 200
```

- 이와 같은 선언을 할 경우 아래의 그림과 같이 print_counter()는 외부의 전역변수 counter를 사용하므로 **counter = 200**을 적용하면 전역변수의 값이 200으로 변경된다.

global 키워드



함수의 내부에서 전역변수를 사용하기 위해서는 global 키워드를 사용합니다. 함수의 내부에서 counter 값을 변경하면 전역변수의 값이 변경됩니다.

지역변수가 아닌 전역변수 counter를 사용하게 된다. 함수 내부에서 값을 바꾸면 외부에서도 그 값이 바뀌게 된다

주의 : 전역변수를 너무 많이 사용하면 코드가 길어질 경우에러발생의 원인이 되니 꼭 필요한 경우에 한해서만 사용합시다.

```
def print_counter():  
    global counter          # 함수  
    counter = 200  
    print('counter =', counter)  
  
counter = 100  
print_counter()  
print('counter =', counter)
```

global 키워드

area 값을 반환하지 않아도
외부에서 그 값이 0에서
78.5로 바뀐다

```
def calculate_area(radius):  
    global area  
    area = 3.14 * radius**2  
    return
```

전역변수 area를 사용함
반환하는 값이 없음

```
area = 0  
r = float(input("원의 반지름을 입력하세요 : "))  
calculate_area(r)  
print(area)
```

```
원의 반지름을 입력하세요 : 5.0  
78.5
```

디폴트 인자

- 파이썬에서는 함수의 매개변수가 기본값을 가질 수 있다. 이것을 **디폴트 인자** `default argument`라고 한다. 예를 들어 보자. 다음과 같이 햄버거 주문 함수 `order()`가 있다고 하자. `greet()`는 항상 3개의 인자를 받아야 한다. 첫 인자는 햄버거의 개수, 두 번째는 피클 포함여부, 세 번째는 양파 포함 여부이다.

```
def order(num, pickle, onion) :  
    print('햄버거 {0} 개 - 피클 {1}, 양파 {2}'.format(num, pickle, onion))  
  
order(1, False, True)
```

```
햄버거 1 개 - 피클 False, 양파 True
```

- 만약 우리가 `order()` 함수에 3개의 인자를 전달하지 않으면 오류가 발생

```
>>> order(1)      # 인자를 1개만 전달하여 order()을 호출함  
...  
TypeError: order1() missing 2 required positional arguments: 'pickle' and 'onion'
```


디폴트 인자

- 이와 같이 인자가 부족한 경우에 기본값을 넣어주는 메커니즘이 있다면 편리할 것이다. 바로 이러한 목적으로 사용하는 것이 디폴트 인자이다.

```
def order(num, pickle = True, onion = True) :  
    print('햄버거 {0} 개 - 피클 {1}, 양파 {2}'.format(num, pickle, onion))  
  
order(1, pickle = False, onion = True)  
order(2)      # 햄버거 2개를 주문, 디폴트로 pickle, onion 값은 True임
```

```
햄버거 1 개 - 피클 False, 양파 True  
햄버거 2 개 - 피클 True, 양파 True
```

- 위의 코드를 살펴보면 pickle과 onion 매개변수에 True라는 디폴트 인자를 넣어 주었다. 따라서 order(2)와 같이 호출해도 프로그램은 오류 없이 잘 수행된다. 반면 이 프로그램에서는 num에 대한 디폴트 값을 주지 않았으며 따라서 order()과 같은 방식으로 함수를 호출하면 프로그램은 오류를 출력할 것이다.

디폴트 인자

- 파이썬에서 대부분의 인자들은 함수 호출 시에 위치에 의하여 구별된다. 예를 들어서 `power(2, 10)`은 `power(10, 2)`과는 다르다. 함수 호출 `power(2, 10)`은 **밑**base이 2이고 **지수**exponent가 10인 **2¹⁰**을 계산할 것이고 `power(10, 2)`은 **10²**을 계산할 것이다.
- 혼동을 줄이고 안전한 프로그램을 위해 **키워드 인자**keyword argument는 인자들 앞에 키워드를 두어서 인자들을 구분한다

```
def power(base, exponent):  
    return base**exponent    # base가 밑이고, exponent가 지수값이다.
```

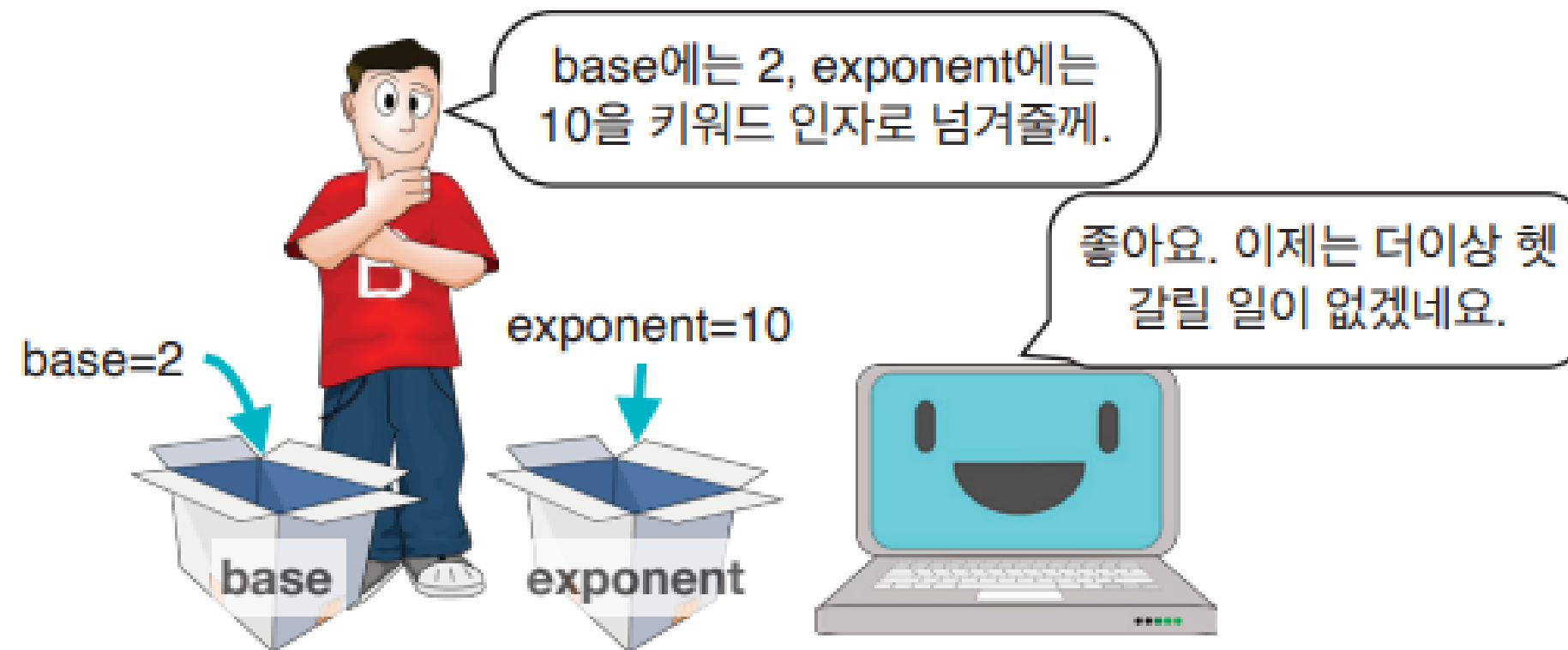
```
>>> power(2, 10)    # 2의 10승을 반환한다  
1024  
>>> power(10, 2)   # 10의 2승을 반환한다  
100
```

어떤 값이 밑이고 지수인지
헷갈릴 수 있다 -> 오류의
원인이 된다

디폴트 인자

```
>>> power(base=2, exponent=10)  # 2의 10승을 반환한다
1024
>>> power(exponent=10, base=2)  # 2의 10승을 반환한다
1024
```

밑(base)과
지수(exponent)를 명시
해주므로 헛갈리지 않는다.



디폴트 인자

- 하지만 다음과 같이 첫 인자에 키워드 인자를 넣고 두번째로 위치 인자로 넣게 되면 오류가 발생한다.
- 인자의 순서는 언제나 **위치 인자가 먼저 나오고 키워드 인자가 나타나야만** 한다.

```
>>> power(base = 10, 2)    # 문법 오류  
SyntaxError: positional argument follows keyword argument
```

재귀 함수

- **재귀함수** *recursion*란 함수 내부에서 자기 자신을 호출하는 함수를 말한다. 재귀는 매우 유용한 문제해결 기법으로 절차적 기법으로 해결하기 어려운 문제를 직관적이고 간단하게 해결할 수 있다.
- 팩토리얼 식과 정의 : $n! = n * (n - 1)!$

재귀적 정의

```
1! = 1
2! = 2 * 1
3! = 3 * 2 * 1
...
n! = n * (n-1) * (n-2) * ... * 2 * 1
```


재귀 함수

- 이 정의를 바탕으로 팩토리얼은 $1! = 1$ 일 때 $n! = n * (n-1)!$ 로 정의할 수 있다. 이를 코드로 구현하면 다음과 같이 나타낼 수 있다.

```
def factorial(n):                # n!의 재귀적 구현
    if n <= 1 :                  # 종료 조건이 반드시 필요하다
        return 1
    else :
        return n * factorial(n-1) # n * (n-1)! 정의에 따른 구현

print('4! = ', factorial(4))    # 인자로 4를 넣어 호출
```

- 재귀함수에서 중요한 점은 이 프로그램이 언제 종료되는가인데 정의에 의하여 $1! = 1$ 인 경우는 그 값이 정의되지만 1이하의 값에 대해서는 팩토리얼을 정의할 필요가 없으므로 if 문에서 n이 1이하가 되면 1을 반환하면 된다.

모듈

- **모듈** `module`이란, 파이썬 함수나 변수 또는 클래스들을 모아놓은 스크립트 파일을 말한다. 파이썬은 여러 기관과 개발자들에 의해서 만들어진 많은 모듈이 있으며 우리는 이 모듈을 이용하여 효과적으로 소프트웨어를 개발할 수 있다.
- 다음은 현재 스크립트나 대화창에서 모듈을 불러오는 명령이다.

```
import 모듈이름1 [, 모듈이름2, ... ]
```

모듈

- 파이썬 설치와 함께 제공되는 모듈을 **파이썬 표준 라이브러리** `python standard library`라고도 하며 문자열과 텍스트 처리, 이진 데이터 처리, 날짜와 시간 처리, 배열 등의 자료형 처리, 수치 연산과 수학 함수, 파일과 디렉토리 접근, 유닉스 시스템의 데이터베이스 접근, 데이터 압축, 그래픽 등을 위한 100여 가지 이상의 모듈이 표준 라이브러리들로 제공된다.
- 우선 날짜와 시간을 다루는 데에 편리한 `datetime` 모듈을 살펴 보자. 다음과 같이 불러들여 사용할 수 있다.

```
>>> import datetime                # 날짜와 시간을 다루는 모듈
>>> datetime.datetime.now()
datetime.datetime(2024, 1, 2, 6, 57, 27, 904565)
```

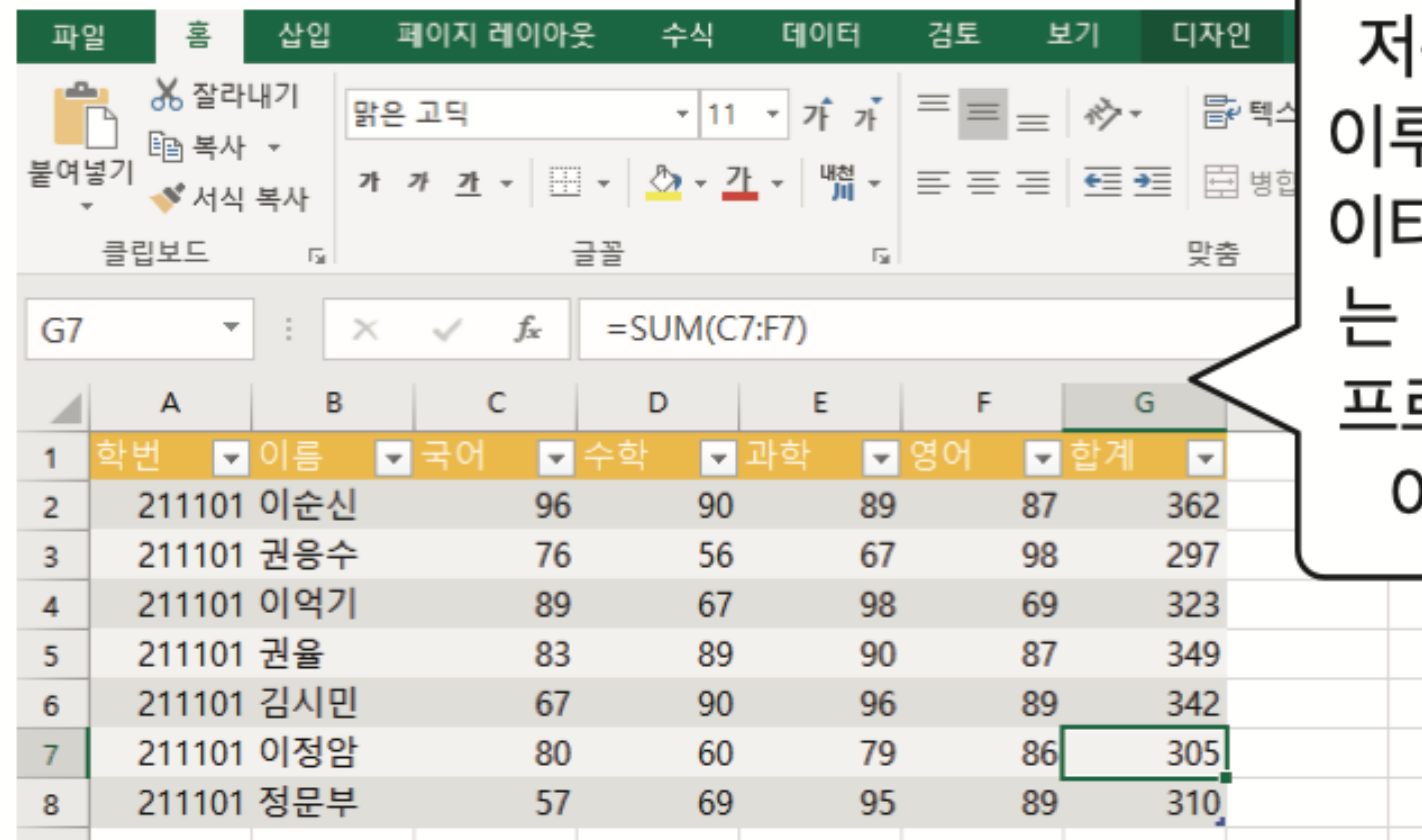

모듈

```
>>> today = datetime.date.today()
>>> print(today)
2023-09-14
>>> today
datetime.date(2023, 9, 14)
>>> today.year
2023
>>> today.month
9
>>> today.day
14
```

Pandas

엑셀보다 빠른 일처리는 판다스로

- **엑셀** Excel은 행과 열로 이루어진 표에 입력된 데이터를 처리하는데 탁월한 성능을 보이고 있다.
- 앞장에서 살펴본 넘파이가 2차원 **행렬** matrix 형태의 데이터를 지원하지만, 넘파이는 데이터의 속성을 표시하는 행이나 열의 레이블을 가지고 있지 않다는 한계가 있다. 하지만 파이썬의 **판다스** pandas 패키지를 사용하면 이러한 문제를 해결할 수 있다.



The screenshot shows the Microsoft Excel interface. The formula bar displays the formula `=SUM(C7:F7)` for cell G7. The spreadsheet contains the following data:

	A	B	C	D	E	F	G
1	학번	이름	국어	수학	과학	영어	합계
2	211101	이순신	96	90	89	87	362
3	211101	권응수	76	56	67	98	297
4	211101	이억기	89	67	98	69	323
5	211101	권율	83	89	90	87	349
6	211101	김시민	67	90	96	89	342
7	211101	이정암	80	60	79	86	305
8	211101	정문부	57	69	95	89	310

저는 행과 열로 이루어진 표의 데이터를 잘 처리하는 스프레드시트 프로그램인 엑셀이라고 해요!

pandas



판다스는 엑셀같은 스프레드시트 프로그램보다 연산을 빠르고 효율적으로 할 수 있어요.

엑셀보다 빠른 일처리는 판다스로

- 판다스로 어떤 일을 할 수 있나
 - 파이썬 리스트, 딕셔너리, 넘파이 배열을 데이터 프레임으로 변환할 수 있다.
 - 판다스로 CSV 파일이나, 엑셀 파일 등을 열 수 있다.
 - URL을 통해 웹 사이트의 CSV 또는 JSON과 같은 원격 파일 또는 데이터베이스를 열 수 있다.



엑셀보다 빠른 일처리는 판다스로

- 판다스로 어떤 일을 할 수 있나
 - 데이터 보기 및 검사
 - mean()로 모든 열의 평균을 계산할 수 있다.
 - corr()로 데이터 프레임의 열 사이의 상관 관계를 계산할 수 있다.
 - count()로 각 데이터 프레임 열에서 null이 아닌 값의 개수를 계산할 수 있다.
 - 필터, 정렬 및 그룹화
 - sort_values()로 데이터를 정렬할 수 있다.
 - 조건을 사용하여 열을 필터링할 수 있다.
 - groupby()를 이용하여 기준에 따라 몇 개의 그룹으로 데이터를 분할할 수 있다.
 - 데이터 정제
 - 데이터의 누락 값을 확인할 수 있다.
 - 특정한 값을 다른 값으로 대체할 수 있다.
-

엑셀보다 빠른 일처리는 판다스로

- 판다스로 어떤 일을 할 수 있나



잠깐 - 판다스 or 판다

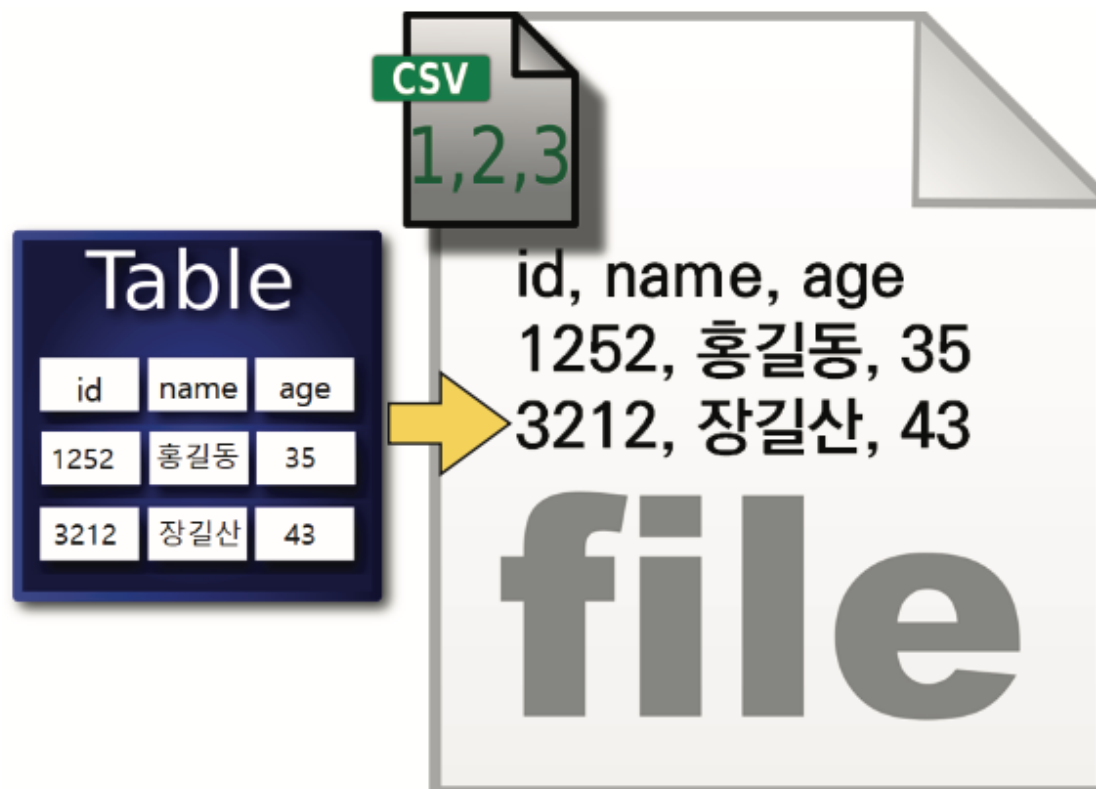
판다스라는 특이한 이름은 "panel data"라는 용어에서 유래되었다. 이 panel data라는 용어 역시 생소한 용어인데 이는 **계량경제학** **econometrics** 용어로 동일한 관찰자에 의하여 여러 회에 걸쳐 관측된 데이터 집합을 지칭하는 용어이다. 중국 쓰촨성일대에 서식하는 동물인 판다와는 관계가 없으나 용어가 비슷하므로 많은 사람들이 판다스의 로고로 판다 그림을 사용하기도 한다.



판다스는 판다
하고 상관없는
데이터 분석
라이브러리에요!

CSV

- CSV는 테이블 형식의 데이터를 저장하고 이동하는 데 사용되는 구조화된 텍스트 파일 형식이다. CSV는 **쉼표로 구분한 변수** comma separated variables의 약자이다.
- CSV의 역사는 1972년으로 거슬러 올라가며 Microsoft Excel와 같은 **스프레드 시트** spread sheet 소프트웨어에 적합한 형식이다. 데이터 과학에서 사용되는 데이터 가운데 상당한 비율의 데이터들이 CSV 형식으로 공유되는 경우가 많다.



CSV

- CSV 파일은 필드를 나타내는 열과 레코드를 나타내는 행으로 구성
- 만약 데이터의 중간에 구분자가 포함되어야 한다면 따옴표를 사용하여 필드를 묶어야 함
 - 예를 들어서 'Gildong, Hong'이라는 데이터가 있다고 하자. 데이터의 중간에 쉼표(,)가 포함되어 있다. 이러한 경우에는 구분자로 사용되는 쉼표와 구분하기 위하여 반드시 데이터를 따옴표로 감싸야 한다.
- CSV 파일의 첫 번째 레코드에는 열 제목이 포함되어 있을 수 있다.
 - CSV 형식 자체의 요구사항이 아니라 단순히 일반적인 관행
- CSV 파일의 크기를 알 수 없고 잠재적으로 크기가 큰 경우 한 번에 모든 레코드를 읽지 않는 것이 좋다.
 - 이때는 현재 행을 읽고, 현재 행을 처리한 후에 삭제하고 다음 행을 가져오는 방식이 필요할 수도 있다. 아니면 특정한 크기만큼의 데이터를 읽어서 처리한 뒤에, 다음으로 또 그만큼의 크기를 가져오는 방식을 사용할 수도 있을 것이다.

CSV

- CSV 데이터의 내용을 읽어 보자
 - 파이썬 모듈 csv는 CSV reader와 CSV writer를 제공한다. 두 객체 모두 파일 핸들을 첫 번째 매개 변수로 사용한다. 필요한 경우 delimiter 매개 변수를 사용하여 구분자를 제공할 수 있다.
 - 이 파일을 d: 드라이브의 data 폴더에 'weather.csv'로 저장했다고 가정하고, 그러면 이 파일의 경로path는 'd:/data/weather.csv'가 된다.

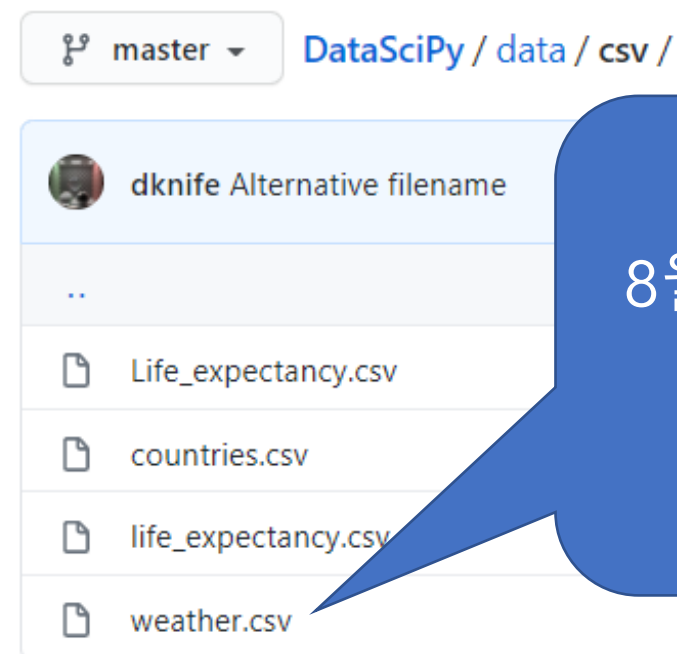
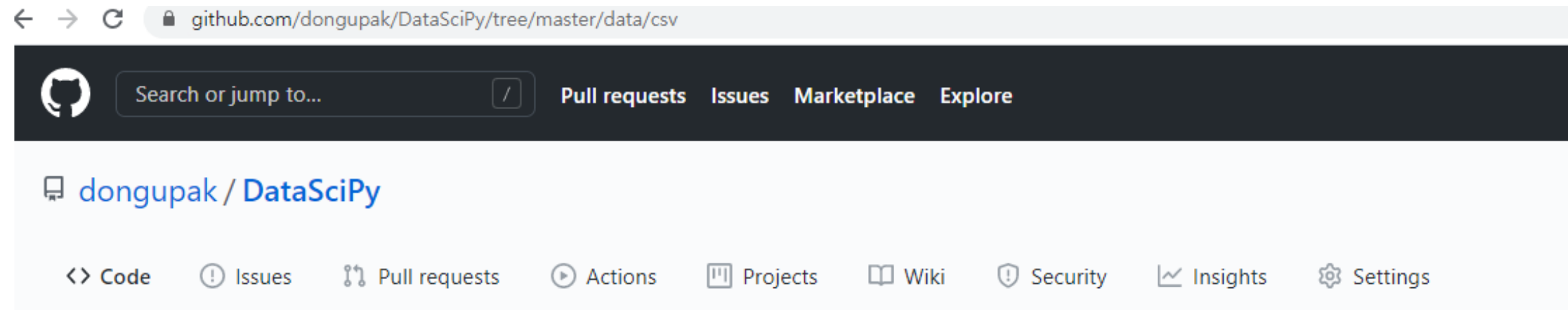
```
import csv      # 판다스가 아닌 파이썬 csv 모듈을 사용함

f = open('d:/data/weather.csv') # CSV 파일을 열어서 f에 저장한다.
data = csv.reader(f)           # reader() 함수를 이용하여 읽는다.
for row in data:
    print(row)
f.close()
```

```
['일시', '평균기온', '최대풍속', '평균풍속']
['2010-08-01', '28.7', '8.3', '3.4']
['2010-08-02', '25.2', '8.7', '3.8']
['2010-08-03', '22.1', '6.3', '2.9']
...
```

CSV

- CSV 데이터의 내용을 읽어 보자

A screenshot of a text editor window showing the content of the 'weather.csv' file. The file contains a table with four columns: '일시' (Date), '평균기온' (Average temperature), '최대풍속' (Maximum wind speed), and '평균풍속' (Average wind speed). The data rows show weather information for the first 12 days of August 2010.

일시	평균기온	최대풍속	평균풍속
2010-08-01	28.7	8.3	3.4
2010-08-02	25.2	8.7	3.8
2010-08-03	22.1	6.3	2.9
2010-08-04	25.3	6.6	4.2
2010-08-05	27.2	9.1	5.6
2010-08-06	26.8	9.8	8
2010-08-07	27.5	9.1	5
2010-08-08	26.6	5.9	4
2010-08-09	26.9	5.1	3.1
2010-08-10	25.6	10.2	5.5
2010-08-11	24.6	9.4	4.8
2010-08-12	23.7	8.7	2.6

CSV

- CSV 데이터의 내용을 읽어 보자
 - 헤더를 제거하는 방법
 - next() 함수를 사용함

```
import csv          # 판다스가 아닌 파이썬 csv 모듈을 사용함

f = open('d:/data/weather.csv') # CSV 파일을 열어서 f에 저장한다.
data = csv.reader(f)           # csv의 reader() 함수를 이용하여 읽는다.
header = next(data)            # 헤더를 제거한다.
for row in data:                # 반복 루프를 사용하여 데이터를 읽는다.
    print(row)
f.close()                    # 파일을 닫는다.
```

```
['2010-08-01', '28.7', '8.3', '3.4']
['2010-08-02', '25.2', '8.7', '3.8']
['2010-08-03', '22.1', '6.3', '2.9']
['2010-08-04', '25.3', '6.6', '4.2']
...
```

CSV

- 기상자료개방 포털 사이트에서 다운 받은 데이터 weather.csv를 계속해서 사용해 보자.
- 이제 이 데이터에서 평균 풍속 데이터만 추출하여 사용하고 싶다. CSV 파일에서 평균 풍속 데이터는 4번째 열에 저장되어 있다. 인덱스로는 3이 된다. 따라서 리스트에서 row[3]을 찾으면 된다

	col[0]	col[1]	col[2]	col[3]
	A	B	C	D
1	일시	평균기온	최대풍속	평균풍속
row	2010-08-01	28.7	8.5	3.4
3	2010-08-02	25.2	8.7	3.8
4	2010-08-03	22.1	6.3	2.9
5	2010-08-04	25.3	6.0	4.2
6	2010-08-05	27.2	9.1	5.6
7	2010-08-06	26.8	9.8	8

```
import csv
```

```
f = open('d:/data/weather.csv')
```

```
data = csv.reader(f)
```

```
header = next(data)
```

```
for row in data:
```

```
    print(row[3], end=',')
```

```
f.close()
```

CSV 파일을 열어서 f에 저장한다.

reader() 함수를 이용하여 읽는다.

헤더를 제거한다.

반복 루프를 사용하여 데이터를 읽는다.

평균풍속만 출력하고, 쉼표로 연결한다.

파일을 닫는다.

```
3.4,3.8,2.9,4.2,5.6,8,5,4,3.1,5.5,4.8,2.6,4.6,4.4,10.3,3.2,1.6,2.1,1.9,3.2,4.2,2.5,6.2,3,1.9,2.5,1.6,2.3,4.9,6.2,4.2,2.6,5.3,1.7,3.2,3.3,4.3,7.6,6.6,2.5,7.2,3.8,1.8,3.9,1.6,2.2,1.2,2.2,3,3.5,2.8,3.6,7.9,5.8,4.1,6.1,1.8,2.8,5.6,2.1,2.2,3.3,3.2,5.9,5,5.1,3.1,3.4,3.7,2.7,2.6,3.1,2.5,5,3.2,2.9,4.5,2.9,2.9,2.1,3.9,6.3,3.9,2,3,6.1,7.1,4,3.5,5.8,6.6,7.2,5.6,3.5,3.2,2.9,3.2,3.3,2.5,7.5 ...
```

CSV

- 반복문을 사용하여 import한 데이터의 세 번째 열의 원소값 중에서 최대 값을 구하자.

```
# 위의 코드 import .. 부터 header =.. 까지가 생략되었음
max_wind = 0.0

for row in data:
    if row[2] == '' :
        wind = 0
    else :
        wind = float(row[2])
        if max_wind < wind :
            max_wind = wind

print('지난 10년간 울릉도의 최대 풍속은 ', max_wind, 'm/s')
```

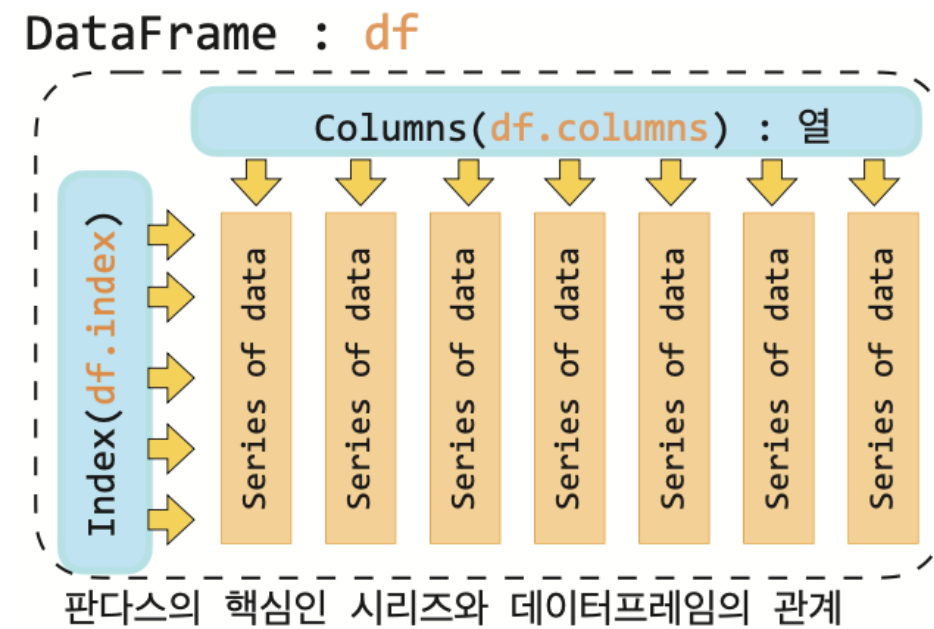
지난 10년간 울릉도의 최대 풍속은 26.0 m/s

시리즈와 데이터프레임

- 앞서 다루어본 csv 모듈 이외에도 CSV 데이터를 처리할 수 있는 모듈이 있다. 이들 중 가장 강력한 외부 라이브러리인 판다스를 알아보자.
- 판다스는 데이터 저장을 위하여 다음과 같은 2가지의 기본 데이터 구조를 제공하고 있다.
- 이들 데이터 구조는 모두 넘파이 배열을 이용하여 구현된다. 따라서 속도가 빠르다. 모든 데이터 구조는 값을 변경할 수 있으며, 시리즈를 제외하고는 크기도 변경할 수 있다. 각 행과 열은 이름이 부여되며, 행의 이름을 **인덱스**`index`, 열의 이름을 **컬럼스**`columns`라 부른다.

데이터 구조	차원	설명
시리즈	1	레이블이 붙어있는 1차원 벡터
데이터프레임	2	행과 열로 되어있는 2차원 테이블, 각 열은 시리즈로 되어 있다.

시리즈와 데이터프레임



Series

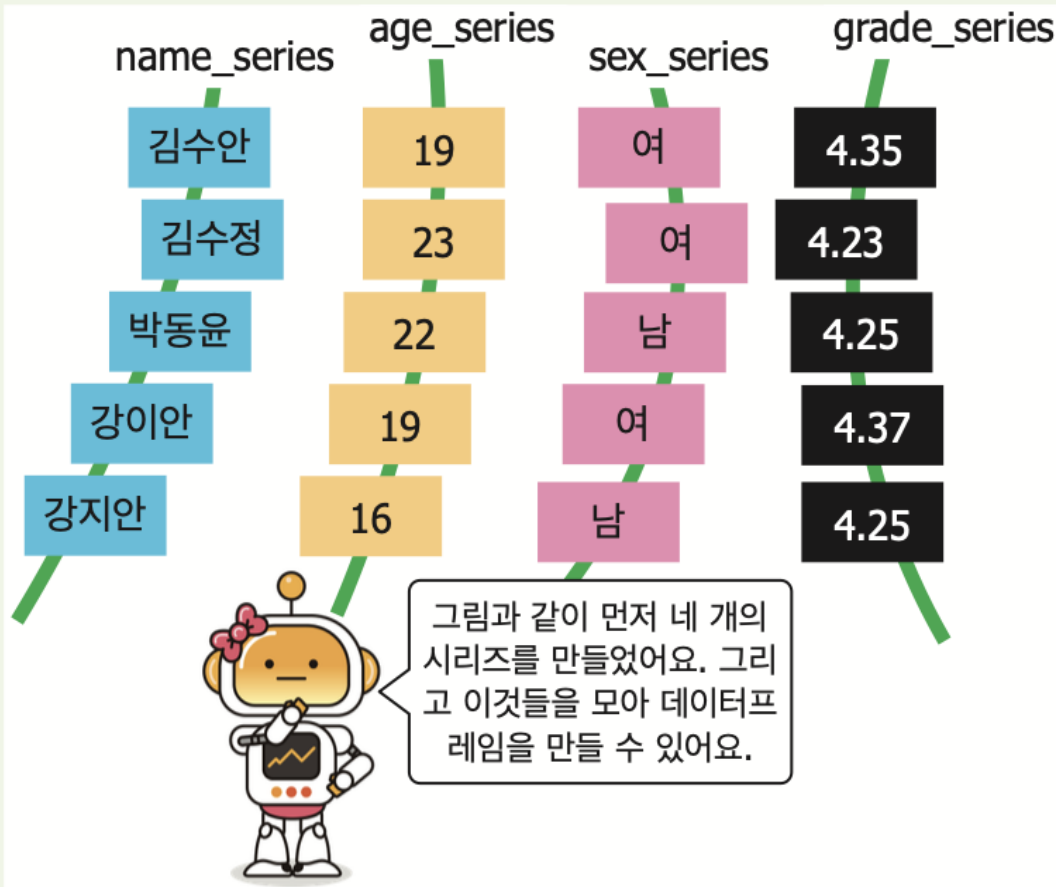
1	3	4	NaN	6	8
---	---	---	-----	---	---

```
>>> import numpy as np
>>> import pandas as pd
>>> series = pd.Series([1, 3, 4, np.nan, 6, 8])
>>> series
0    1.0
1    3.0
2    4.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

np.nan은 값 없음(Not a Number)을 의미하는 것으로
수치 데이터가 없을 경우 이를 표기하는 방법이다.
NaN과 동일한 표기임.
nan이 있을 경우 실수형

시리즈와 데이터프레임

```
>>> name_series = pd.Series(['김수안', '김수정', '박동윤', '강이안', '강지안'])
>>> age_series = pd.Series([19, 23, 22, 19, 16])
>>> sex_series = pd.Series(['여', '여', '남', '여', '남'])
>>> grade_series = pd.Series([4.35, 4.23, 4.25, 4.37, 4.25])
>>> print(name_series, age_series, sex_series, grade_series)
0    김수안
1    김수정
2    박동윤
3    강이안
4    강지안
dtype: object
0    19
1    23
2    22
3    19
4    16
dtype: int64
0    여
1    여
2    남
3    여
4    남
dtype: object
0    4.35
1    4.23
2    4.25
3    4.37
4    4.25
dtype: float64
```



이름	나이	성별	평점
김수안	19	여	4.35
김수정	23	여	4.23
박동윤	22	남	4.45
강이안	19	여	4.37
강지안	16	남	4.25

시리즈와 데이터프레임

딕셔너리 형식의 데이터로
데이터 프레임을 생성함

```
>>> df = pd.DataFrame({'이름': name_series, '나이': age_series,  
                        '성별': sex_series, '평점': grade_series})
```

```
>>> print(df)
```

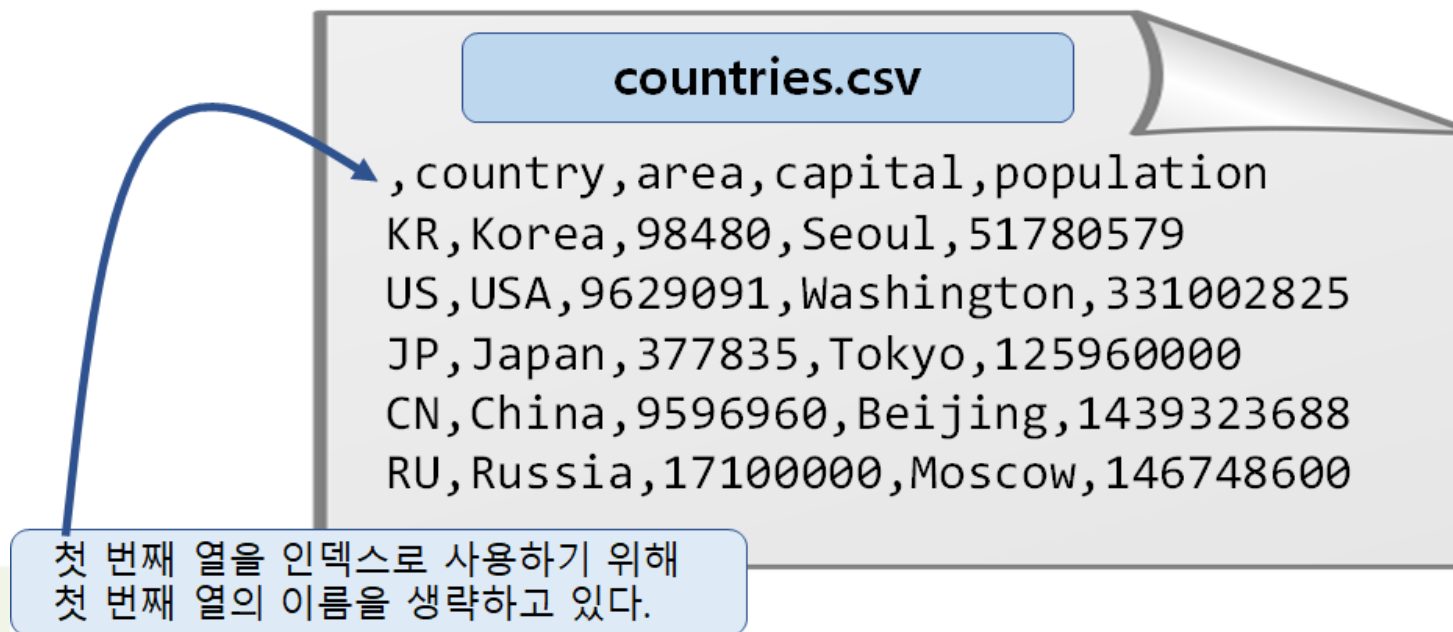
	이름	나이	성별	평점
0	김수안	19	여	4.35
1	김수정	23	여	4.23
2	박동윤	22	남	4.25
3	강이안	19	여	4.37
4	강지안	16	남	4.25

	columns			
	이름	나이	성별	평점
Values	김수안	19	여	4.35
	김수정	23	여	4.23
	박동윤	22	남	4.25
	강이안	19	여	4.37
	강지안	16	남	4.25

판다스의 DataFrame 클래스를
사용해서 하나의 데이터 프레임을
만들 수 있다

판다스로 데이터 파일 읽기

- 판다스 모듈을 이러한 csv 파일을 읽어들이어서 데이터프레임으로 바꾸는 작업을 간단히 할 수 있게 한다. 다음과 같이 `read_csv` 함수를 이용하면 된다. `countries.csv` 파일의 제 1행 제 1열은 비어 있음을 확인할 수 있다. 이것은 첫 열은 데이터가 아니라 각 행의 인덱스로 사용되도록 하기 위해서이다.
- 이때 CSV 파일이 데이터프레임이 될 수 있도록 각 행이 같은 구조로 되어 있고, 각 열은 동일한 자료형을 가진 시리즈로 되어 있어야 한다. 예러가 없이 csv 파일을 읽어왔다면 `df`를 출력해보자.



```
>>> import pandas as pd
>>> df = pd.read_csv('d:/data/countries.csv')
```

판다스로 데이터 파일 읽기

```
>>> df
   Unnamed: 0 country  area  capital  population
0           KR  Korea   98480    Seoul    51780579
1           US   USA  9629091 Washington  331002825
2           JP  Japan   377835    Tokyo   125960000
3           CN  China  9596960    Beijing  1439323688
4           RU  Russia 17100000    Moscow   146748600
```

각 열은 서로 다른 속성 레이블을 나타낸다.

인덱스 번호는 판다스가 추가한 열이다

판다스로 데이터 파일 읽기

- 데이터를 설명하는 인덱스와 컬럼스 객체

비워 두었던 열 이름

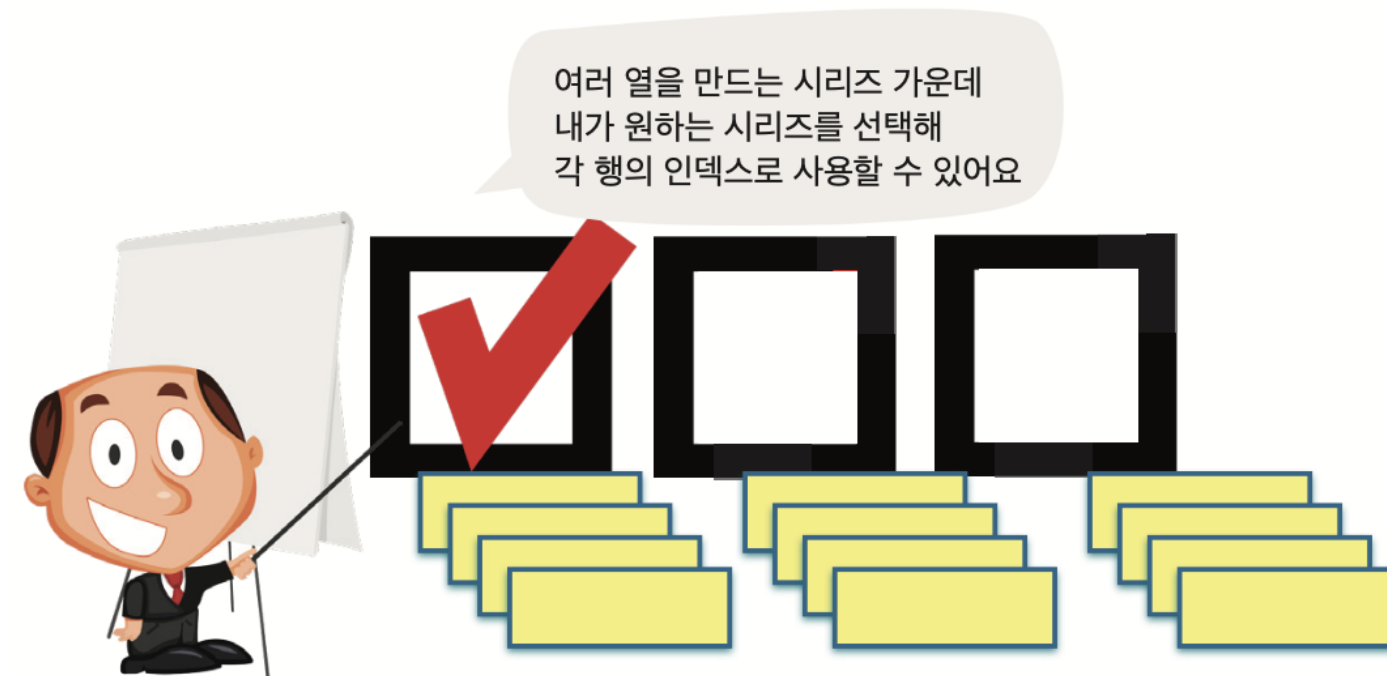
CSV 파일의 첫 행으로 만들어진 **columns**

	Unnamed: 0	country	area	capital	population
0	KR	Korea	98480	Seoul	51780579
1	US	USA	9629091	Washington	331002825
2	JP	Japan	377835	Tokyo	125960000
3	CN	China	9596960	Beijing	1439323688
4	RU	Russia	17100000	Moscow	146748600

자동으로 생성된 **index**

판다스로 데이터 파일 읽기

- 데이터를 설명하는 인덱스와 컬럼스 객체



```
import pandas as pd
```

```
df = pd.read_csv('d:/data/countries.csv', index_col = 0) # 첫 열을 인덱스 컬럼으로 사용  
print(df)
```

	country	area	capital	population
KR	Korea	98480	Seoul	51780579
US	USA	9629091	Washington	331002825
JP	Japan	377835	Tokyo	125960000
CN	China	9596960	Beijing	1439323688
RU	Russia	17100000	Moscow	146748600

열을 기준으로 데이터 선택하기

- 특정한 열만 선택하려면 아래와 같이 대괄호 안에 열의 이름을 넣으면 된다.
- 다음 코드는 countries.csv를 다시 읽고 있다. 그리고 처음에는 인덱스를 첫 열로 지정해서 df_my_index로 할당했고, 인덱스 지정 없이 만든 데이터프레임은 df_no_index로 할당했다. 두 데이터프레임에서 population 레이블을 가진 열을 추출하기 위해서는 df['population']이라고 하면 된다.

```
import pandas as pd

df_my_index = pd.read_csv('d:/data/countries.csv', index_col = 0)
df_no_index = pd.read_csv('d:/data/countries.csv')
print(df_my_index['population'])
print(df_no_index['population'])
```

```
KR      51780579
US      331002825
JP      125960000
CN      1439323688
RU      146748600
Name: population, dtype: int64
0      51780579
1      331002825
2      125960000
3      1439323688
4      146748600
Name: population, dtype: int64
```

인덱스 컬럼이 0이므로 KR, US, JP,..가 인덱스가 된다

인덱스 컬럼이 없을 경우 0, 1, 2, ..가 인덱스가 됨

열을 기준으로 데이터 선택하기

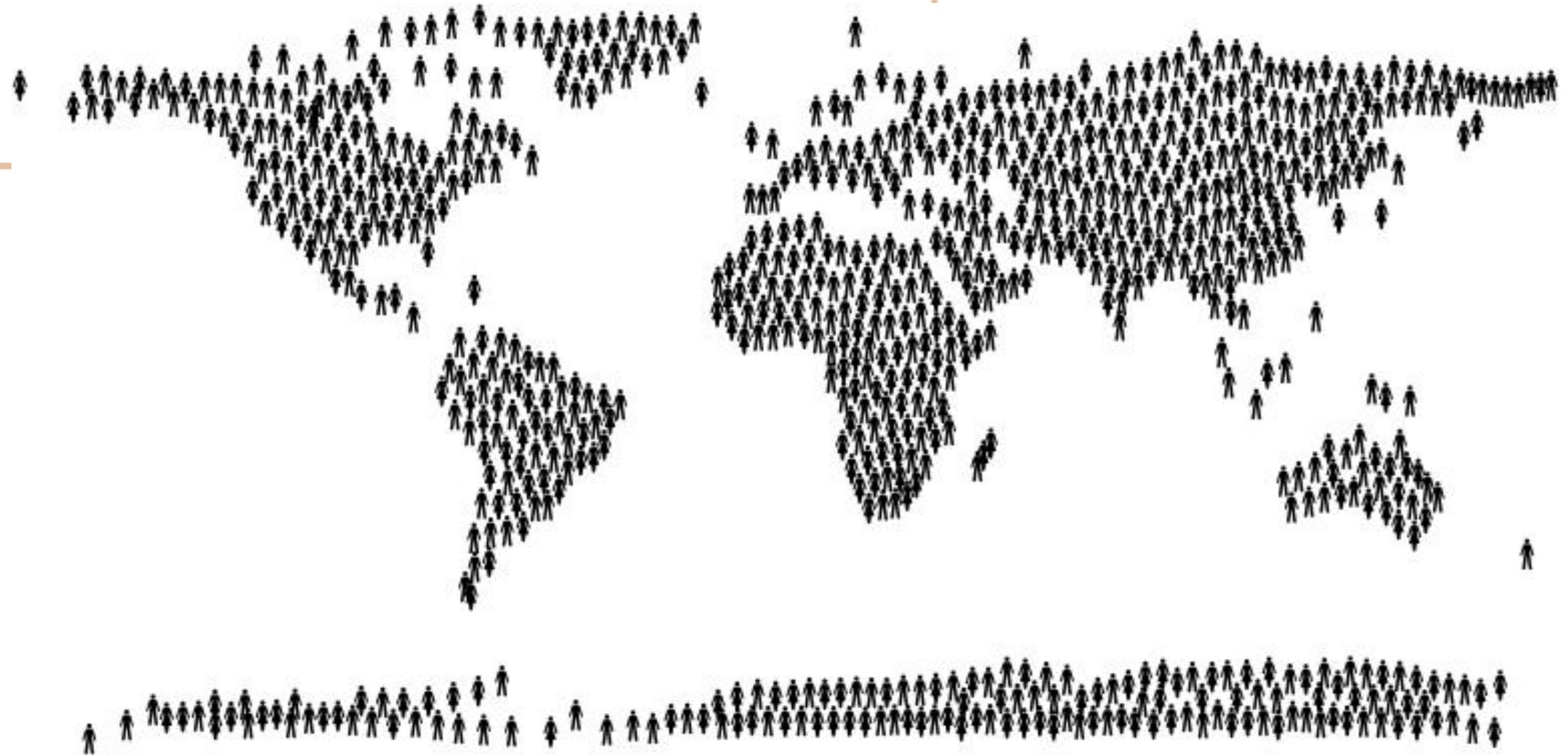
```
import pandas as pd
```

```
df_my_index = pd.read_csv('d:/data/countries.csv', index_col = 0)
```

```
print(df_my_index[ ['area', 'population'] ])
```

	area	population
KR	98480	51780579
US	9629091	331002825
JP	377835	125960000
CN	9596960	1439323688
RU	17100000	146748600

전체 데이터 중에서 두 개의 열을
선택하는 경우 : 선택을 원하는 열의
레이블을 리스트에 넣어서 전달



열을 기준으로 데이터 선택하기

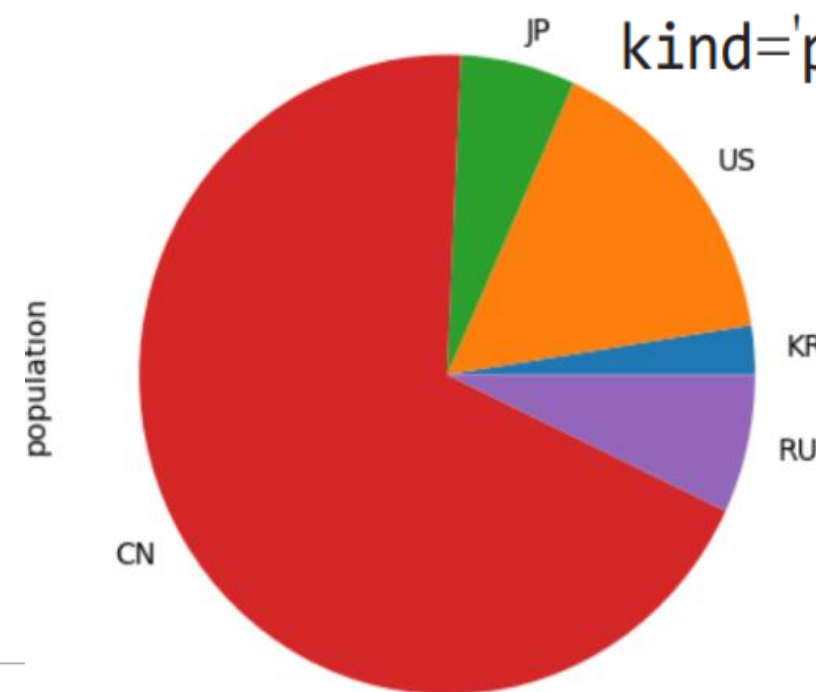
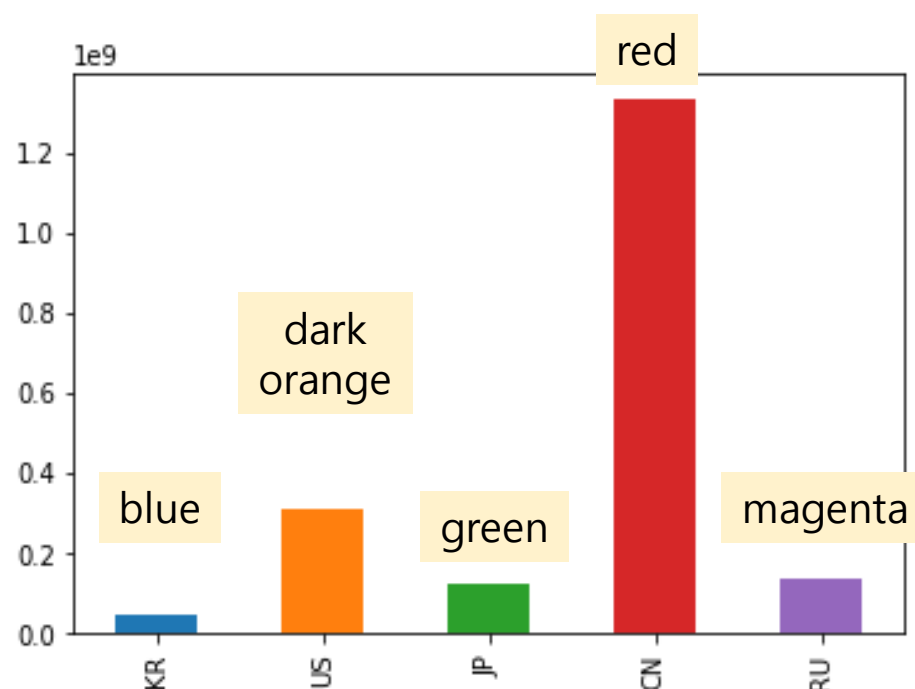
- 데이터 시각화하기

- 우리는 선택된 열을 그래프로 그릴 수 있다. 이를 위하여 데이터 프레임의 이름 다음에 plot() 메소드만 추가하면 된다. 각 국가의 인구만을 추출하여서 막대 그래프로 그려보면 다음과 같다.

```
import pandas as pd
import matplotlib.pyplot as plt

countries_df = pd.read_csv('d:/data/countries.csv', index_col = 0)

countries_df['population'].plot(kind='bar', color=('b', 'darkorange', 'g', 'r', 'm'))
plt.show()
```



kind='pie'로 지정하고, color 부분을 삭제

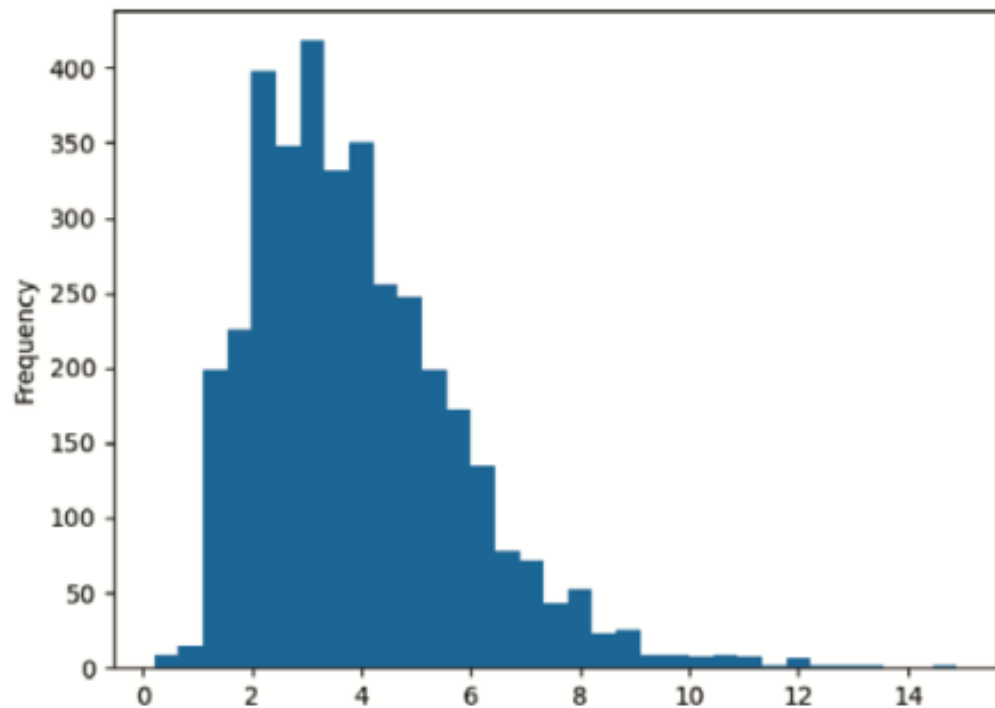
열을 기준으로 데이터 선택하기

• 데이터 시각화하기

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
weather = pd.read_csv('d:/data/weather.csv', index_col = 0, encoding='CP949')
weather['평균풍속'].plot(kind='hist', bins=33)
plt.show()
```

한글 인코딩 문자를
읽어오기 위해서 사용함



판다스를 이용하면 데이터
를 차트로 표시하는 일을
아주 간편하게 할 수 있답
니다.



슬라이딩으로 행 선택하기

- 데이터 프레임 중에서 몇 개의 행만을 가져오고자 할 때는 몇 가지의 방법이 있다. 우선 처음 5행만 얻으려면 `head()`를 사용할 수 있다. 마지막 5행만을 얻으려면 `tail()`을 사용한다.

```
>>> countries_df.head()                                     # countries_df[0:5]와 같다
```

	country	area	capital	population
KR	Korea	98480	Seoul	48422644
US	USA	9629091	Washington	310232863
JP	Japan	377835	Tokyo	127288000
CN	China	9596960	Beijing	1330044000
RU	Russia	17100000	Moscow	140702000

```
>>> countries_df[:3]
```

	country	area	capital	population
KR	Korea	98480	Seoul	48422644
US	USA	9629091	Washington	310232863
JP	Japan	377835	Tokyo	127288000

슬라이싱으로 행 선택하기

```
>>> countries_df.loc['KR']  
country      Korea  
area         98480  
capital      Seoul  
population   48422644
```

행의 레이블이 'KR'인 행만을 선택하기

```
>>> countries_df['population'][:3]  
KR      48422644  
US      310232863  
JP      127288000
```

```
>>> countries_df.loc['US', 'capital']  
'Washington'
```

데이터프레임에서 특정한 요소 하나만을
선택하려면 loc 속성에 행과 열의 레이블을
써주면 된다

```
>>> countries_df['capital'].loc['US']  
'Washington'
```

새로운 열 생성하기

- 새로운 열을 생성해 보자
 - 판다스를 이용하면 다른 열의 정보를 토대로 새로운 열을 생성할 수도 있다.
 - 우리의 데이터프레임에 인구 밀도를 나타내는 열을 생성해보자. 앞장에서 넘파이 배열에는 어떤 수를 곱하고 더하는 것이 가능하다고 하였다. 판다스는 넘파이를 기반으로 하기 때문에 판다스 데이터 프레임에도 동일하게 적용할 수 있다. 인구를 면적으로 나눠주면 된다.

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
countries_df = pd.read_csv('d:/data/countries.csv', index_col = 0)
countries_df['density'] = countries_df['population'] / countries_df['area']
print(countries_df)
```

	country	area	capital	population	density
KR	Korea	98480	Seoul	51780579	525.797918
US	USA	9629091	Washington	331002825	34.375293
JP	Japan	377835	Tokyo	125960000	333.373033
CN	China	9596960	Beijing	1439323688	149.977044
RU	Russia	17100000	Moscow	146748600	8.581789

countries_df['density'] 열이
새롭게 추가되었음

새로운 열 생성하기

- 새로운 열을 생성해 보자

인구밀도는 인구를 면적으로 나누면 됩니다. 판다스는 이와 같은 방식으로 새로운 열을 추가하는 것도 가능합니다. **for** 반복문을 사용하는 것 보다 간결하며 수행 속도도 빠릅니다.



인구밀도 = 인구 / 면적

`df['density'] = df['population'] / df['area']`

	인구	면적	인구밀도
			NEW!
			NEW!
			NEW!
			NEW!

데이터 분석

- 데이터 프레임이 저장한 데이터를 간단히 분석하려면 `describe()` 함수를 호출해주면 된다.

```
import pandas as pd
weather = pd.read_csv('d:/data/weather.csv', index_col = 0, encoding='CP949')

print(weather.describe())
```

	평균기온	최대풍속	평균풍속
count	3653.000000	3649.000000	3647.000000
mean	12.942102	7.911099	3.936441
std	8.538507	3.029862	1.888473
min	-9.000000	2.000000	0.200000
25%	5.400000	5.700000	2.500000
50%	13.800000	7.600000	3.600000
75%	20.100000	9.700000	5.000000
max	31.300000	26.000000	14.900000

데이터 분석

```
...  
print('평균 분석 -----')  
print(weather.mean())  
print('표준편차 분석 -----')  
print(weather.std())
```

```
평균 분석 -----  
평균기온      12.942102  
최대풍속      7.911099  
평균풍속      3.936441  
dtype: float64  
표준편차 분석 -----  
평균기온      8.538507  
최대풍속      3.029862  
평균풍속      1.888473  
dtype: float64
```

데이터 분석

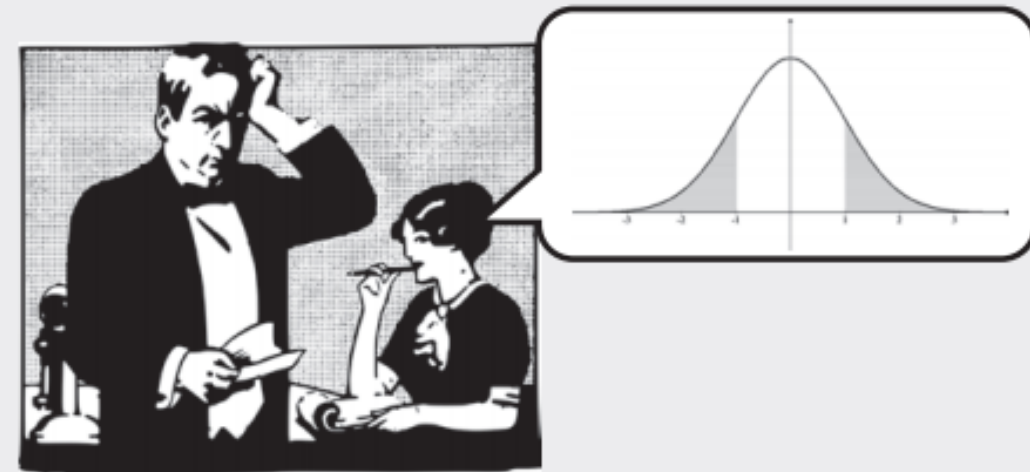


잠깐 - 판다스의 표준편차와 넘파이의 표준편차의 차이

평균기온 표준편차를 넘파이로 계산하면 다른 값이 나온다. 판다스 표준편차와 넘파이 표준편차를 각각 구하는 방법은 다음과 같다.

```
pandas_std = weather['평균기온'].std()  
numpy_std = np.std( weather['평균기온'] )  
print(pandas_std, numpy_std)
```

```
8.538507014753446 8.537338236838895
```



판다스의 표준편차는 디폴트로 **베셀 보정**Bessel's correction을 적용하는데, 이것은 표준편차를 구할 때, 표본 크기 n 대신에 $n-1$ 을 적용하는 것으로 모분산 추정에서 편향을 보정하는 역할을 한다.

데이터 분석

- 데이터의 전체적인 특징이 어떠한지를 분석하는 것은 매우 중요한 일이다. 앞서 살펴본 `describe()` 함수는 데이터프레임의 데이터 특성을 전체적으로 요약해 주는데, 이 분석 내용 각각은 하나씩 떼어서 적용할 수도 있다.
- 다음 코드는 데이터의 수를 알려준다

```
>>> weather = pd.read_csv('d:/data/weather.csv', index_col = 0, encoding='CP949')
>>> weather.count()    # 각 열의 데이터 수를 알려준다
평균기온      3653
최대풍속      3649
평균풍속      3647
dtype: int64
```

weather.csv 파일이 담고 있는 데이터가
3 개의 열을 가지고 있고, 각각의 열에 담긴
데이터가 3653, 3649, 3647개라는 것을 알 수 있다

```
>>> weather['최대풍속'].count()    # '최대풍속' 열의 데이터 수를 알려준다
3649
```

데이터 분석

- 데이터 집계 분석도 쉽게 해 보자 (계속)

```
>>> weather[['최대풍속', '평균풍속']].count() # '최대풍속', '평균풍속' 열의 데이터 수
최대풍속      3649
평균풍속      3647
dtype: int64
```

여러 개의 열을 분석하고 싶을 때는
원하는 열의 레이블들을 리스트로
제공

```
>>> weather[['최대풍속', '평균풍속']].mean() # '최대풍속', '평균풍속' 열의 평균값
최대풍속      7.911099
평균풍속      3.936441
dtype: float64
```

min(), max(), mean(), sum() 등도
적용 가능

```
>>> weather.mean()[['최대풍속', '평균풍속']]
최대풍속      7.911099
평균풍속      3.936441
dtype: float64
```



필터링

- weather 데이터프레임에서 '최대풍속' 레이블로 되어 있는 열의 값이 10.0을 넘는지 확인하여 참과 거짓을 얻는 방법은 다음과 같다.

```
>>> weather['최대풍속'] >= 10.0
```

일시

2010-08-01 False

2010-08-02 False

2010-08-03 False

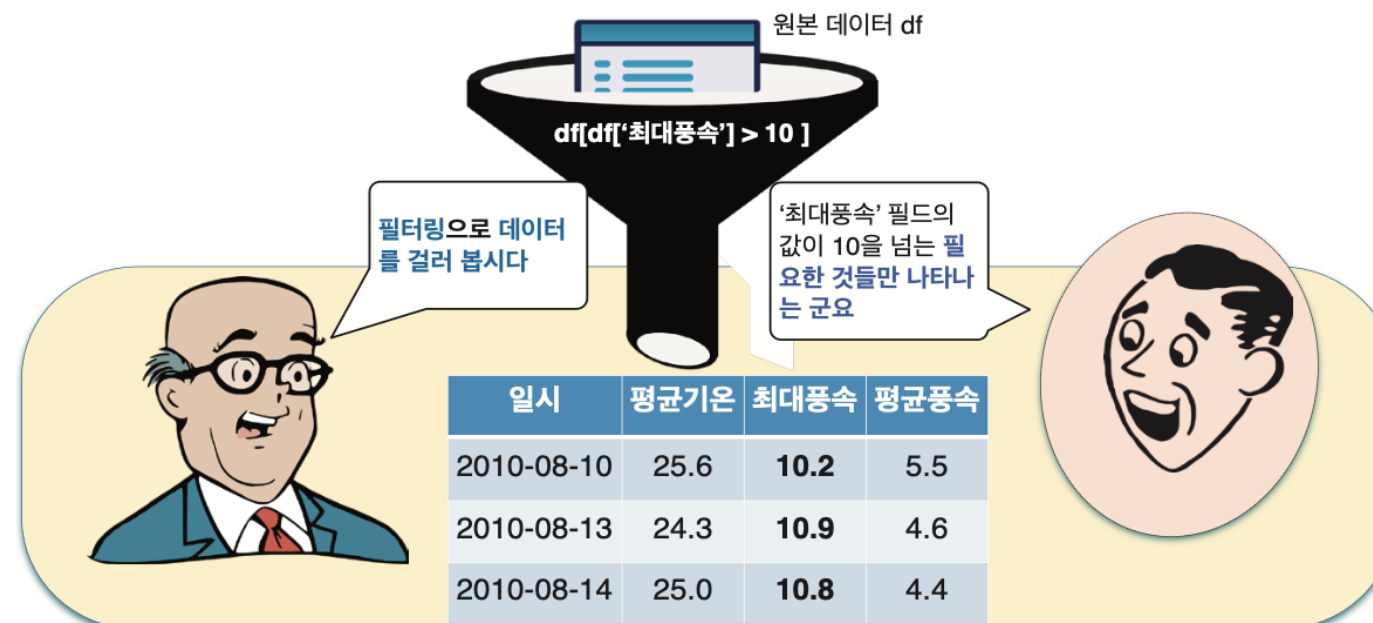
...

2020-07-29 False

2020-07-30 False

2020-07-31 False

넘파이의 논리 인덱싱과
동일한 문법



필터링

```
>>> weather[ weather['최대풍속'] >= 10.0 ]
```

평균기온 최대풍속 평균풍속

일시

2010-08-10	25.6	10.2	5.5
2010-08-13	24.3	10.9	4.6
2010-08-14	25.0	10.8	4.4
...	...	...	...
2020-07-13	17.8	10.3	4.6
2020-07-14	17.8	12.7	9.4
2020-07-20	23.0	11.2	7.3

'최대풍속' 레이블로 되어 있는 열의 값이
10.0을 넘는지 확인하여 참값을 얻는 방법

필터링

- 빠진 값을 찾고 삭제하기

- 데이터 과학자가 사용하는 실제 데이터는 완벽하지 않고 상당한 수의 결손값을 가지고 있거나 의심스러운 값을 가지고 있다.
 - 결손값은 데이터가 아예 수집되지 않았거나, 측정 장치의 고장, 사건 사고 등으로 데이터를 확보할 수 없는 경우 생길 수 있다.
 - 따라서 데이터를 처리하기 전에 반드시 거쳐야 하는 절차가 데이터 정제이다.
 - 판다스에서는 결손값을 NaN으로 나타낸다(혹은 NA로 표기함).
 - 판다스는 결손값missing data을 탐지하고 수정하는 함수를 제공한다.
-

필터링

- 빠진 값을 찾고 삭제하기
 - weather.csv 데이터 역시 이러한 결손값이 존재한다. 데이터에 결손값이 있는지를 확인하는 함수는 isna()이다. 평균풍속이 측정되지 않았는지를 다음과 같이 확인해 보자.

```
weather['평균풍속'].isna()
```



필터링

- 빠진 값을 찾고 삭제하기
 - `weather[ weather['평균풍속'].isna() ]`라고 하면, 이 조건을 이용하여 데이터프레임의 일부를 가져올 것이다. 즉, 평균 풍속이 측정되지 않은 날들만 추출해 보고 싶다면 아래 코드로 가능하다. 지난 10년의 데이터 가운데 평균풍속이 기록되지 않은 날은 6일임을 알 수 있다.

```
import pandas as pd
```

```
weather = pd.read_csv('d:/data/weather.csv', index_col = 0, encoding='CP949')  
missing_data = weather [ weather['평균풍속'].isna() ]  
print(missing_data)
```

	평균기온	최대풍속	평균풍속
일시			
2012-02-11	-0.7	NaN	NaN
2012-02-12	0.4	NaN	NaN
2012-02-13	4.0	NaN	NaN
2015-03-22	10.1	11.6	NaN
2015-04-01	7.3	12.1	NaN
2019-04-18	15.7	11.7	NaN

필터링

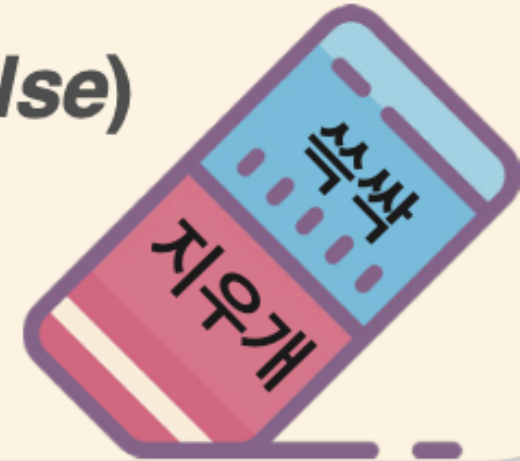
- 빠진 값을 찾고 삭제하기

축이 0이면 결손데이터를 포함한 행을 삭제하고
축이 1이면 결손데이터를 포함한 열을 삭제한다.

inplace가 True이면 원본 데이터에서 결손데이터를 삭제하고
False인 경우는 원본은 그대로 두고 고쳐진 데이터프레임 반환

pandas.DataFrame.dropna(*axis=0*, *how='any'*, *inplace=False*)

how의 값이 'any'이면 결손 데이터가 하나라도 포함되면 제거 대상이 되고,
'all'이면 axis 인자에 따라서 행 혹은 열 전체가 결손 데이터이어야 제거한다.



```
>>> weather.dropna(axis=0, how="any", inplace=True)
>>> weather.loc['2012-02-11']    # 2012년 2월 11일 데이터가 삭제되어 오류발생
... raise KeyError(key) from err
KeyError: '2012-02-11'
```


빠진 데이터 메우기

- 우리가 사용하고 있는 weather.csv의 결손값을 새로운 값으로 채워보자. 아래의 코드는 fillna() 함수를 이용하여 결손값을 0으로 채우고 있다. 그리고 이러한 작업이 원본에 반영되도록 inplace=True로 설정했음을 유의해서 보자. 평균풍속의 결손값이 존재했던 2012년 2월 11일의 데이터를 출력해 보자. 결손값 NaN이 아니라 0이 채워진 것을 확인할 수 있다.

```
import pandas as pd
```

```
weather = pd.read_csv('d:/data/weather.csv', index_col = 0, encoding='CP949')
```

```
weather.fillna(0, inplace = True) # 결손값을 0으로 채움, inplace를 True로 설정해 원본 데이터를 수정
```

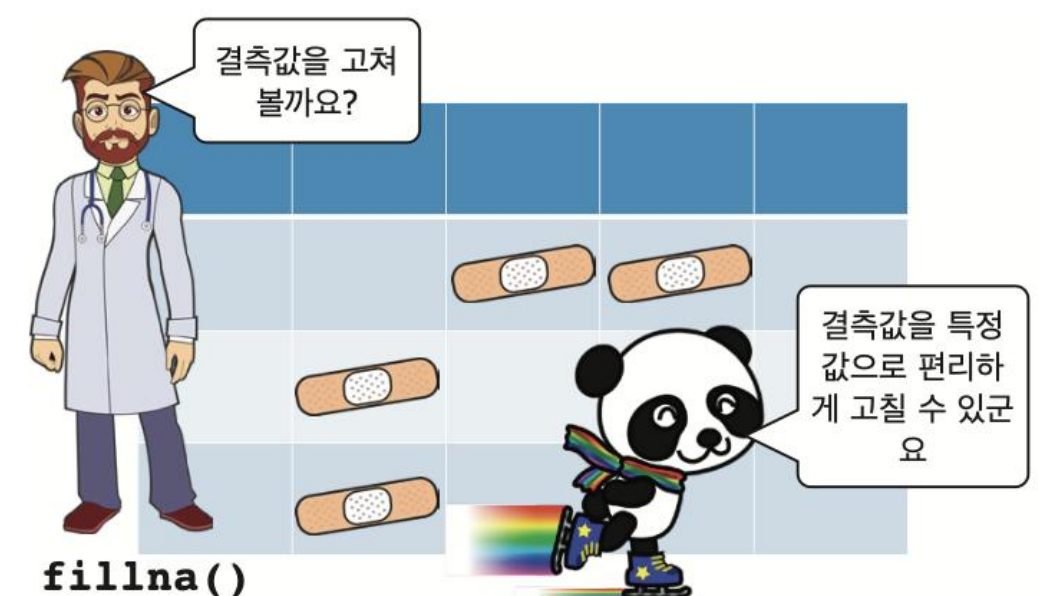
```
print(weather.loc['2012-02-11'])
```

평균기온 -0.7

최대풍속 0.0

평균풍속 0.0

Name: 2012-02-11, dtype: float64



빠진 데이터 메우기



잠깐 – inplace 매개변수

판다스 모듈은 데이터프레임에 대한 조작을 하는 다양한 함수를 제공한다. 그리고 많은 함수들이 `inplace` 매개변수를 가지고 있다. 이것은 조작이 원본 데이터에 이루어지는 것인지, 아니면 사본을 만들어 사본을 변경하는지를 결정하게 된다. 판다스를 다룰 때 흔히 범하는 실수는 `inplace`의 디폴트 인자가 `False`임을 잘 모르고, 원본 데이터프레임이 변경되었다고 생각하는 것이다.

아래 코드를 실행하면 여전히 `NaN`이 출력될 것이다.

```
weather.fillna(0)
print(weather.loc['2012-02-11'][2])
```

```
weather.fillna( weather['평균풍속'].mean(), inplace = True)
print(weather.loc['2012-02-11'])    # 2012년 2월 11일 데이터가 삭제되지 않음
```

```
평균기온    -0.700000
최대풍속     3.936441
평균풍속     3.936441
Name: 2012-02-11, dtype: float64
```

정렬

- 데이터 분석에 있어서 우리가 가지고 있는 데이터를 크기에 따라 순서대로 나열하는 정렬sort은 필수적이다. 정렬은 판다스에서 그리 어렵지 않다. `sort_values()` 함수를 호출하면 된다. 이 함수의 인자로 정렬할 열의 이름과 오름차순, 내림차순 결과를 지정할 수 있다.

```
import pandas as pd
import matplotlib.pyplot as plt

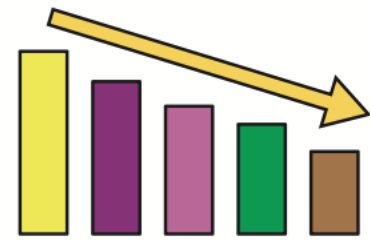
countries_df = pd.read_csv('d:/data/countries.csv', index_col = 0)
sorted_df = countries_df.sort_values('population') # 인구 수(population)의 오름차순 정렬
print(sorted_df)
```

	country	area	capital	population
KR	Korea	98480	Seoul	51780579
JP	Japan	377835	Tokyo	125960000
RU	Russia	17100000	Moscow	146748600
US	USA	9629091	Washington	331002825
CN	China	9596960	Beijing	1439323688

정렬

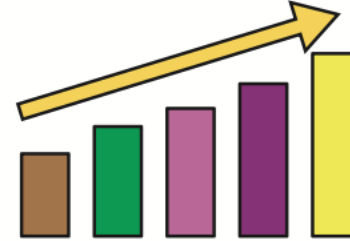
```
countries_df = pd.read_csv('d:/data/countries.csv', index_col = 0)
countries_df.sort_values('population', inplace = True)
print(countries_df)
```

ascending = False



내림차순 정렬

ascending = True



오름차순 정렬



정렬은 데이터 분석의 가장 기본 기능입니다. 오름차순 정렬을 위해서는 ascending 인자를 True로 지정합니다.

```
countries = pd.read_csv('d:/data/countries.csv', index_col = 0, encoding='CP949')
countries.sort_values(['population', 'area'], ascending = False, inplace = True)
print(countries)
```

	country	area	capital	population
CN	China	9596960	Beijing	1439323688
US	USA	9629091	Washington	331002825
RU	Russia	17100000	Moscow	146748600
JP	Japan	377835	Tokyo	125960000
KR	Korea	98480	Seoul	51780579

데이터 구조 변경

- 데이터프레임을 csv를 읽어서 생성할 수도 있지만, 딕셔너리 데이터를 이용하여 생성할 수도 있다. 이때 키는 열의 레이블이 되고, 딕셔너리의 키에 딸린 값은 열을 채우는 데이터를 가진 리스트가 된다.
- 딕셔너리의 한 항목이 시리즈 데이터가 되는 것이다. 다음과 같은 방식으로 데이터프레임을 만들어 보자.

```
import pandas as pd

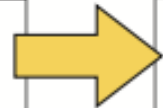
df_1 = pd.DataFrame({'item' : ['ring0', 'ring0', 'ring1', 'ring1'],
                     'type' : ['Gold', 'Silver', 'Gold', 'Bronze'],
                     'price': [20000, 10000, 50000, 30000]})
```

	item	type	price
0	ring0	Gold	20000
1	ring0	Silver	10000
2	ring1	Gold	50000
3	ring1	Bronze	30000

데이터 구조 변경

index	item	type	price
0	ring0	Gold	20000
1	ring0	Silver	10000
2	ring1	Gold	50000
3	ring1	Bronze	30000

pivot 이전의 데이터프레임



item	Bronze	Gold	Silver
ring0	NaN	20000	10000
ring1	30000	50000	NaN

pivot 이후의 데이터프레임



데이터프레임의
pivot() 기능을 사용하
여 테이블의 구조를 변
경할 수 있습니다.

```
df_2 = df_1.pivot(index='item', columns='type', values='price')  
print(df_2)
```

```
type    Bronze    Gold    Silver  
item  
ring0      NaN  20000.0  10000.0  
ring1  30000.0  50000.0      NaN
```


데이터 구조 변경

- concat() 함수로 데이터프레임을 합쳐보자
 - 일반적으로 데이터들은 하나의 큰 테이블로 저장되지 않고, 작은 테이블로 나누어져 있는 경우가 많다. 이것은 저장과 관리의 편의성 때문이기도 하고, 데이터 수집의 시기, 주체 등이 달라 별도로 생성된 경우가 많기 때문이다. 이 절에서는 이러한 데이터를 하나로 합치는 방법 가운데 하나인 concat() 함수를 살펴보자.
 - 우선 다음과 같이 데이터프레임을 두 개 준비해 보자.
 - 다음과 같이 키:값의 딕셔너리 데이터를 이용하여 만들 수 있고, index를 원하는 값으로 설정할 수 있다.

```
df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],  
                      'B' : ['b10', 'b11', 'b12'],  
                      'C' : ['c10', 'c11', 'c12']} , index = ['가', '나', '다'] )  
  
df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],  
                      'C' : ['c23', 'c24', 'c25'],  
                      'D' : ['d23', 'd24', 'd25']} , index = ['다', '라', '마'] )
```

데이터 구조 변경

- concat() 함수로 데이터프레임을 합쳐보자

df_1

	A	B	C
가	a10	b10	c10
나	a11	b11	c11
다	a12	b12	c12

인덱스

df_2

	B	C	D
다	b23	c23	d23
라	b24	c24	d24
마	b25	c25	d25

인덱스

합칠 데이터프레임의 리스트

축이 0이면 테이블의 행을 늘려서 붙여 나감
축이 1이면 테이블의 열을 늘리며 붙여 나감

pandas.concat(df_list, axis=0, join='outer')

join 매개변수는 테이블들을 붙일 때 레이블들을 어떻게 사용할지 결정한다.
이 키워드 인자가 'outer'이면 레이블들의 합집합으로 생성하고 'inner'이면
레이블들의 교집합으로 생성된다.



```
df_3 = pd.concat( [df_1, df_2]) # df_1, df_2 두 데이터프레임을 합쳐서 df_3을 생성
print(df_3)
```

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b23	c23	d23
라	NaN	b24	c24	d24
마	NaN	b25	c25	d25

감사합니다

Jinu Gong
